

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

NGUYỄN ĐĂNG THANH TUỆ

KHÓA LUẬN TỐT NGHIỆP
THIẾT KẾ BỘ VI XỬ LÝ 2 LỖI RISC-V DỰA TRÊN
KIẾN TRÚC TẬP LỆNH RV32IMFA

**Design of a Dual-Core RISC-V Microprocessor Based on the
RV32IMFA Instruction Set Architecture**

CỬ NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

NGUYỄN ĐĂNG THANH TUỆ – 22521616

KHÓA LUẬN TỐT NGHIỆP
THIẾT KẾ BỘ VI XỬ LÝ 2 LỖI RISC-V DỰA TRÊN
KIẾN TRÚC TẬP LỆNH RV32IMFA

**Design of a Dual-Core RISC-V Microprocessor Based on the
RV32IMFA Instruction Set Architecture**

CỬ NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

GIẢNG VIÊN HƯỚNG DẪN
THÂN THẾ TÙNG
TRẦN ĐẠI DƯƠNG

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG CHẤM KHÓA LUẬN TỐT NGHIỆP

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số **710/QĐ-ĐHCNTT**, ngày 19/06/2026 của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến thầy Thân Thế Tùng và thầy Trần Đại Dương đã tận tình hướng dẫn và tạo điều kiện thuận lợi để em hoàn thành khóa luận.

Trong suốt quá trình thực hiện khóa luận, nhờ sự chỉ dẫn nhiệt tình của các thầy, em đã tiếp thu được nhiều kiến thức bổ ích và nhận được những góp ý quý báu để hoàn thiện báo cáo.

Em cũng xin cảm ơn các bạn trong lớp đã động viên, thảo luận và góp ý, giúp em có thêm động lực vượt qua những khó khăn trong quá trình thực hiện đồ án.

Mặc dù đã nỗ lực hoàn thành báo cáo, nhưng do hạn chế về kiến thức và kinh nghiệm, em không tránh khỏi những thiếu sót. Em rất mong nhận được sự thông cảm và góp ý từ thầy để em có thể hoàn thiện hơn trong tương lai. Em xin chân thành cảm ơn.

Thành phố Hồ Chí Minh, Ngày 08 Tháng 07 Năm 2026

Sinh viên thực hiện

Nguyễn Đăng Thanh Tuệ

MỤC LỤC

Chương 1. GIỚI THIỆU ĐỀ TÀI.....	3
1.1. Tổng quan đề tài.....	3
1.2. Mục tiêu đề tài.....	5
1.3. Giới hạn đề tài	5
Chương 2. CƠ SỞ LÝ THUYẾT.....	7
2.1. Định dạng lệnh	7
2.2. Chức năng lệnh.....	8
2.3. Kiến trúc đa lỗi.....	15
Chương 3. THIẾT KẾ HỆ THỐNG	17
3.1. Thiết kế hệ thống tổng quát.....	17
3.2. Thiết kế chi tiết.....	18
3.2.1. Thiết kế phần cứng.....	18
3.2.2. Thiết kế giao tiếp bộ nhớ theo chuẩn AXI4.....	42
Chương 4. XÁC MINH VÀ ĐÁNH GIÁ THIẾT KẾ	49
4.1. Mục tiêu kiểm thử	49
4.2. Môi trường và công cụ kiểm thử.....	50
4.3. Kiểm thử chức năng các nhóm lệnh RV32IMFA	51
4.3.1. Kiểm thử nhóm lệnh RV32I.....	52
4.3.2. Kiểm thử nhóm lệnh RV32M	53
4.3.3. Kiểm thử nhóm lệnh RV32F.....	54
4.3.4. Kiểm thử nhóm lệnh RV32A	55
4.4. Kịch bản kiểm thử đa lỗi.....	56
4.4.1. Kiểm thử truy cập tuần tự	57
4.4.2. Kiểm thử truy cập đồng thời khác địa chỉ.....	57
4.4.3. Kiểm thử truy cập đồng thời cùng địa chỉ.....	58
4.4.4. Kiểm thử thao tác nguyên tử trong môi trường đa lỗi	59
4.5. Triển khai thực nghiệm bộ xử lý RV32IMFA trên kit KV260	61

4.6. Đánh giá hiệu năng hệ thống và kết quả kiểm thử chức năng	61
Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	64
5.1. Kết luận	64
5.2. Hạn chế của đề tài	65
5.3. Hướng phát triển	66
TÀI LIỆU THAM KHẢO	67

DANH MỤC HÌNH

Hình 3.1: Sơ đồ tổng quát hệ thống dual-core RV32IMFA.....	17
Hình 3.2: Sơ đồ chi tiết tầng IF của bộ xử lý RV32IMFA	19
Hình 3.3: Sơ đồ chi tiết tầng ID của bộ xử lý RV32IMFA.....	21
Hình 3.4: Sơ đồ khối tầng thực thi lệnh (EX) của bộ xử lý RV32IMFA.....	27
Hình 3.5: Sơ đồ khối tầng MEM của bộ xử lý RV32IMFA	34
Hình 3.6: Sơ đồ khối tầng WB của bộ xử lý RV32IMFA	37
Hình 3.7: Sơ đồ khối Hazard Unit của bộ xử lý RV32IMFA	39
Hình 3.8: Sơ đồ giao tiếp bộ nhớ dạng AXI4 của hệ thống RV32IMFA	42

DANH MỤC BẢNG

Bảng 2.1: Định dạng lệnh của RISC-V	7
Bảng 2.2: Cấu trúc và chức năng của từng lệnh RISC-V	9
Bảng 3.1: Chức năng các khối trong hệ thống dual-core RV32IMFA	17
Bảng 3.2: Các tín hiệu I/O của tầng IF	19
Bảng 3.3: Các trường của tập lệnh RISC-V 32-bit	20
Bảng 3.4: Các tín hiệu I/O của tầng ID	21
Bảng 3.5: Các tín hiệu vào/ra của khối Register File	23
Bảng 3.6: Các tín hiệu vào/ra của khối FP_Register File	24
Bảng 3.7: Điều kiện hoạt động của khối FP_Register File	24
Bảng 3.8: Các tín hiệu I/O của khối Control Unit	25
Bảng 3.9: Điều kiện tạo tín hiệu ghi thanh ghi trong Control Unit	25
Bảng 3.10: Các tín hiệu I/O của khối Sign Extend	26
Bảng 3.11: Bảng mô tả hoạt động của khối Sign Extend	26
Bảng 3.12: Các tín hiệu I/O của tầng EX	27
Bảng 3.13: Các tín hiệu I/O của khối ALU	29
Bảng 3.14: Mã điều khiển các phép toán trong ALU	29
Bảng 3.15: Các tín hiệu I/O trong FPU	30
Bảng 3.16: Mã điều khiển các phép toán trong FPU	31
Bảng 3.17: Các tín hiệu I/O của khối MDU	32
Bảng 3.18: Mã điều khiển các phép toán trong MDU	32
Bảng 3.19: Các tín hiệu I/O của tầng MEM	34
Bảng 3.20: Cách hoạt động của tầng MEM	35
Bảng 3.21: Các tín hiệu I/O của tầng WB	37
Bảng 3.22: Cách hoạt động của tầng WB	38
Bảng 3.23: Các tín hiệu I/O của khối Hazard Unit	39
Bảng 3.24: Cách hoạt động của khối Hazard Unit	41
Bảng 3.25: Các tín hiệu của kênh địa chỉ đọc AR	43
Bảng 3.26: Các tín hiệu giao tiếp của kênh dữ liệu đọc R	44

Bảng 3.27: Các tín hiệu giao tiếp của kênh địa chỉ ghi AW	44
Bảng 3.28: Các tín hiệu giao tiếp của kênh dữ liệu ghi W	45
Bảng 3.29: Các tín hiệu giao tiếp của kênh phản hồi ghi B.....	46
Bảng 4.1: Kết quả kiểm thử các nhóm chức năng của bộ xử lý RV32IMFA.....	62

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Ý nghĩa
ALU	Arithmetic Logic Unit	Khối số học và logic
AMO	Atomic Memory Operation	Thao tác bộ nhớ nguyên tử
AR	Address Read Channel	Kênh địa chỉ đọc trong giao tiếp AXI
AW	Address Write Channel	Kênh địa chỉ ghi trong giao tiếp AXI
AXI4	Advanced eXtensible Interface 4	Chuẩn giao tiếp AXI phiên bản 4
AXI-Lite	Advanced eXtensible Interface Lite	Phiên bản AXI đơn giản cho giao tiếp thanh ghi
BRAM	Block Random Access Memory	Bộ nhớ khối trên FPGA
CPU	Central Processing Unit	Bộ xử lý trung tâm
CSR	Control and Status Register	Thanh ghi điều khiển và trạng thái
DMEM	Data Memory	Bộ nhớ dữ liệu
DSP	Digital Signal Processing	Khối xử lý tín hiệu số trên FPGA
EX	Execute Stage	Tầng thực thi trong pipeline
FF	Flip-Flop	Phần tử nhớ tuần tự trong FPGA
FPGA	Field-Programmable Gate Array	Mạch tích hợp có thể lập trình
FPU	Floating Point Unit	Khối xử lý số dấu chấm động
HDL	Hardware Description Language	Ngôn ngữ mô tả phần cứng
HEX	Hexadecimal File	Tập tin mã máy ở dạng hệ thập lục phân
ID	Instruction Decode Stage	Tầng giải mã lệnh trong pipeline
IEEE	Institute of Electrical and Electronics Engineers	Viện Kỹ sư Điện và Điện tử
IF	Instruction Fetch Stage	Tầng nạp lệnh trong pipeline
IMEM	Instruction Memory	Bộ nhớ lệnh
ISA	Instruction Set Architecture	Kiến trúc tập lệnh

KV260	Kria KV260 Vision AI Starter Kit	Kit FPGA dùng để triển khai thực nghiệm
LR	Load-Reserved	Lệnh đọc và đặt giữ chỗ địa chỉ bộ nhớ
LUT	Look-Up Table	Bảng tra logic trong FPGA
MDU	Multiply Divide Unit	Khối nhân chia số nguyên
MEM	Memory Access Stage	Tầng truy cập bộ nhớ trong pipeline
NoC	Network on Chip	Mạng kết nối trên chip
PC	Program Counter	Bộ đếm chương trình
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
RF	Register File	Tập thanh ghi
RISC	Reduced Instruction Set Computer	Máy tính với tập lệnh rút gọn
RISC-V	Reduced Instruction Set Computer - Five	Kiến trúc tập lệnh RISC-V mã nguồn mở
RTL	Register Transfer Level	Mức mô tả truyền thanh ghi
RV32A	RISC-V 32-bit Atomic Extension	Mở rộng lệnh nguyên tử 32-bit
RV32F	RISC-V 32-bit Single-Precision Floating-Point Extension	Mở rộng xử lý dấu chấm động đơn chính xác 32-bit
RV32I	RISC-V 32-bit Integer Base ISA	Tập lệnh số nguyên cơ bản 32-bit
RV32M	RISC-V 32-bit Integer Multiplication and Division Extension	Mở rộng nhân chia số nguyên 32-bit
RV32IMFA	RISC-V 32-bit I, M, F, A Extensions	Bộ tập lệnh gồm các mở rộng I, M, F và A
SC	Store-Conditional	Lệnh ghi có điều kiện trong cơ chế LR/SC
SoC	System on Chip	Hệ thống tích hợp trên một chip
Spike	RISC-V ISA Simulator	Trình mô phỏng tham chiếu cho RISC-V
TB	Testbench	Môi trường kiểm thử mô phỏng

Vitis	Xilinx Vitis Unified Software Platform	Môi trường phát triển phần mềm của Xilinx
Vivado	Xilinx Vivado Design Suite	Bộ công cụ thiết kế, tổng hợp và triển khai FPGA
WB	Write Back Stage	Tầng ghi kết quả trong pipeline
WSTRB	Write Strobe	Tín hiệu chọn byte ghi trong kênh dữ liệu ghi AXI

TÓM TẮT KHÓA LUẬN

Khóa luận trình bày quá trình thiết kế và kiểm thử bộ vi xử lý hai lõi dựa trên kiến trúc RISC-V RV32IMFA, tập trung vào khả năng hoạt động song song và phối hợp giữa các lõi trong hệ thống đa lõi. Hai lõi có thể thực thi chương trình độc lập, đồng thời cùng truy cập bộ nhớ dùng chung thông qua cơ chế phân xử nhằm hạn chế xung đột tài nguyên.

Mỗi lõi được xây dựng theo kiến trúc pipeline, hỗ trợ các nhóm lệnh RV32I, RV32M, RV32F và RV32A. Các khối chức năng chính gồm ALU, MDU, FPU, tập thanh ghi, khối điều khiển và cơ chế xử lý hazard. Khi hai lõi phát sinh yêu cầu truy cập bộ nhớ cùng lúc, khối phân xử lựa chọn một lõi được cấp quyền truy cập và giữ lõi còn lại ở trạng thái chờ. Các lệnh nguyên tử cũng được tích hợp để hỗ trợ đồng bộ dữ liệu khi nhiều lõi cùng thao tác trên một vùng nhớ.

Thiết kế được kiểm chứng bằng mô phỏng Verilog trên Vivado, đối chiếu với Spike và triển khai trên kit FPGA KV260. Các chương trình kiểm thử bao gồm từng nhóm lệnh và các tình huống đa lõi như truy cập tuần tự, truy cập đồng thời khác địa chỉ, cùng địa chỉ và thực hiện thao tác nguyên tử.

Kết quả cho thấy hệ thống thực thi đúng các nhóm lệnh đã thiết kế, đồng thời hỗ trợ phân xử và đồng bộ khi hai lõi cùng truy cập bộ nhớ dùng chung. Đây là nền tảng để tiếp tục tối ưu hiệu năng, mở rộng số lõi và phát triển các hệ thống RISC-V đa lõi hoàn chỉnh hơn.

ABSTRACT

This thesis presents the design and verification of a dual-core microprocessor based on the RISC-V RV32IMFA instruction set architecture, focusing on parallel processing and coordination between cores. The system consists of two pipelined cores that can execute programs independently while accessing shared memory through an arbitration mechanism.

Each core supports the RV32I, RV32M, RV32F, and RV32A instruction extensions. Its main functional blocks include the ALU, MDU, FPU, register file, control unit, and hazard handling mechanism. When both cores request memory access simultaneously, the arbiter grants access to one core and places the other in a waiting state to prevent bus conflicts. Atomic instructions are also implemented to support data synchronization in shared memory.

The design is verified using Verilog/Vivado simulations, comparison with the Spike simulator, and implementation on the KV260 FPGA development kit. Test programs cover individual instruction groups and multicore scenarios, including sequential access, simultaneous access to different or identical addresses, and atomic operations.

The results show that the processor correctly executes the supported instructions and effectively manages shared memory access between two cores. This work provides a foundation for future performance optimization, core expansion, and the development of more advanced multicore RISC-V systems.

Chương 1. GIỚI THIỆU ĐỀ TÀI

1.1. Tổng quan đề tài

Trong những năm gần đây, RISC-V đã trở thành một trong những kiến trúc tập lệnh mở được quan tâm rộng rãi trong lĩnh vực thiết kế bộ xử lý và hệ thống nhúng. Với ưu điểm về tính mở, khả năng tùy biến và khả năng mở rộng theo từng nhóm lệnh, RISC-V tạo điều kiện thuận lợi cho việc nghiên cứu, thiết kế và hiện thực các bộ vi xử lý trên nền tảng FPGA. Xu hướng nghiên cứu hiện nay không chỉ dừng lại ở các lõi xử lý đơn lẻ mà còn mở rộng sang các hệ thống đa lõi, nhằm đáp ứng nhu cầu xử lý song song, chia sẻ tài nguyên và tăng hiệu năng tính toán.

Trong các công trình trước đây, Cao Thanh Bình và Đặng Phi Hùng đã thiết kế và hiện thực lõi vi xử lý RISC-V RV32IF trên FPGA, có hỗ trợ bộ nhớ đệm 4-Way Set Associative Cache và bộ tạo số ngẫu nhiên thực [1]. Công trình này cho thấy khả năng mở rộng của lõi RISC-V với các thành phần hỗ trợ bộ nhớ và bảo mật. Tuy nhiên, phạm vi nghiên cứu chủ yếu tập trung vào kiến trúc đơn lõi, chưa giải quyết bài toán phối hợp và chia sẻ tài nguyên giữa nhiều lõi xử lý.

Đỗ Tuấn Thuận đã nghiên cứu hiện thực khối xử lý dấu chấm động và tích hợp vào bộ xử lý RISC-V 32-bit [2]. Thiết kế này là cơ sở quan trọng cho việc mở rộng khả năng tính toán số thực trong bộ xử lý RISC-V, đặc biệt đối với các phép toán dấu chấm động theo chuẩn IEEE 754. Tuy nhiên, công trình vẫn tập trung chủ yếu vào việc tích hợp FPU cho bộ xử lý đơn lõi, chưa mở rộng sang môi trường đa lõi có yêu cầu đồng bộ dữ liệu và xử lý tranh chấp tài nguyên.

Khuu Thành Tài và Phạm Trí Đức đã đề xuất thiết kế bộ vi xử lý hai lõi dựa trên kiến trúc tập lệnh RISC-V RV32ICMFA [3]. Công trình này có ý nghĩa gần với hướng nghiên cứu của đề tài do cùng tiếp cận bài toán thiết kế bộ xử lý RISC-V hai lõi. Tuy nhiên, thiết kế vẫn còn giới hạn ở một số điều kiện kiểm thử và cần tiếp tục được mở rộng trong việc đánh giá tài nguyên, cơ chế giao tiếp giữa các lõi và khả năng hiện thực trên phần cứng FPGA.

Nguyễn Phan Hoàng Phú đã nghiên cứu thiết kế bộ xử lý đa lõi dựa trên kiến trúc tập lệnh RISC-V [4]. Công trình này cho thấy tiềm năng của RISC-V trong việc xây dựng các hệ thống xử lý nhiều lõi, đồng thời đặt nền tảng cho việc nghiên cứu các cơ chế chia sẻ tài nguyên và đồng bộ giữa các lõi. Tuy vậy, việc tích hợp đầy đủ các mở rộng tập lệnh như M, F và A vẫn là hướng cần tiếp tục phát triển để đáp ứng tốt hơn các bài toán tính toán số nguyên, số thực và đồng bộ bộ nhớ.

Trương Trần Phương Nam và Nguyễn Đức Duy Khang đã hiện thực hệ thống NoC sử dụng lõi vi xử lý RISC-V RV32IMF thông qua giao thức TileLink trên FPGA [5]. Công trình này cho thấy vai trò quan trọng của giao thức kết nối trong việc mở rộng hệ thống xử lý và tổ chức trao đổi dữ liệu giữa các thành phần. Tuy nhiên, nghiên cứu vẫn chưa tập trung sâu vào việc triển khai và kiểm chứng hoạt động của hệ thống hai lõi RV32IMFA với các tình huống đồng bộ dữ liệu thông qua tập lệnh nguyên tử.

Về mặt nền tảng lý thuyết, kiến trúc RISC-V được xây dựng theo hướng mô-đun, trong đó tập lệnh cơ sở RV32I có thể được mở rộng bằng các nhóm lệnh như M, F và A để hỗ trợ nhân/chia số nguyên, xử lý dấu chấm động và thao tác nguyên tử [6]. Đối với các phép toán dấu chấm động, chuẩn IEEE 754 là cơ sở quan trọng để định nghĩa cách biểu diễn số thực, các phép toán cơ bản và các điều kiện ngoại lệ trong quá trình xử lý [7]. Ngoài ra, các giao thức kết nối như TileLink cũng được sử dụng trong nhiều hệ thống RISC-V nhằm hỗ trợ trao đổi dữ liệu giữa lõi xử lý, bộ nhớ và các ngoại vi [8].

Các tài liệu nền tảng về tổ chức máy tính và kiến trúc RISC-V cũng cung cấp cơ sở quan trọng cho quá trình thiết kế đường dữ liệu, bộ điều khiển, pipeline và cơ chế xử lý hazard [9, 10]. Bên cạnh đó, các nghiên cứu về RISC-V mã nguồn mở, Rocket Chip, OpenPiton+Ariane và các kiến trúc RISC-V đa lõi cho thấy xu hướng mở rộng từ bộ xử lý đơn lõi sang hệ thống nhiều lõi có khả năng chia sẻ tài nguyên và mở rộng hiệu năng [11-15].

Qua phân tích các công trình liên quan, có thể thấy phần lớn các nghiên cứu trước đây tập trung vào một số hướng riêng lẻ như thiết kế lõi RISC-V đơn lõi, tích hợp khối xử lý dấu chấm động, xây dựng cơ chế kết nối hoặc mở rộng sang kiến trúc

đa lỗi. Tuy nhiên, việc kết hợp các mở rộng RV32IMFA trong một bộ vi xử lý hai lỗi, đồng thời xem xét cơ chế thực thi pipeline, xử lý hazard, hỗ trợ phép toán dấu chấm động và thao tác nguyên tử vẫn còn là một hướng nghiên cứu cần được tiếp tục phát triển. Vì vậy, đề tài này hướng tới thiết kế bộ vi xử lý lỗi kép dựa trên kiến trúc tập lệnh RV32IMFA, nhằm xây dựng một mô hình xử lý có khả năng thực thi các nhóm lệnh số nguyên, nhân/chia, dấu chấm động và nguyên tử, đồng thời làm cơ sở cho việc mở rộng và đánh giá các hệ thống RISC-V đa lỗi trên FPGA.

1.2. Mục tiêu đề tài

Thiết kế và hiện thực bộ xử lý đa lỗi dựa trên kiến trúc tập lệnh RISC-V RV32IMFA, tích hợp giao thức kết nối AXI4. Thiết kế nhằm tối ưu hóa khả năng giao tiếp bộ nhớ, đáp ứng yêu cầu về hiệu năng tính toán số thực, và hỗ trợ cho các bài toán đồng bộ mức phần mềm. Từ mục tiêu tổng quát, nhóm chia thành 4 mục tiêu chi tiết như sau:

- Hiểu kiến trúc tập lệnh RISC-V IMFA và giao thức AXI4.
- Thiết kế được bộ vi xử lý có thể thực thi các mở rộng MFA và ổn định ở tần số 70MHz.
- Tích hợp được hệ thống 2 lõi vi xử lý với giao thức AXI4 truy cập bộ nhớ chính xác và hỗ trợ mở rộng A (Atomic) hiệu quả.
- Thiết kế hoạt động đúng chức năng và có thể triển khai trên FPGA.

1.3. Giới hạn đề tài

Trong phạm vi khóa luận, bộ vi xử lý chỉ hỗ trợ một phần các chức năng thuộc kiến trúc tập lệnh RISC-V RV32IMFA. Đối với tập lệnh cơ sở RV32I, thiết kế hỗ trợ 37 chỉ thị phục vụ các phép toán số nguyên, truy cập bộ nhớ, rẽ nhánh và điều khiển luồng chương trình. Các chức năng thuộc kiến trúc đặc quyền như hệ thống thanh ghi điều khiển và trạng thái CSR, các lệnh ECALL, EBREAK và Privileged ISA chưa được triển khai.

Đối với phần mở rộng RV32F, bộ xử lý hỗ trợ 26 chỉ thị số chấm động đơn chính xác. Thiết kế chỉ sử dụng chế độ làm tròn mặc định và chưa hỗ trợ việc thay

đổi chế độ làm tròn trong quá trình thực thi. Các thanh ghi trạng thái dấu chấm động, cờ ngoại lệ và cơ chế xử lý ngoại lệ số chấm động cũng không nằm trong phạm vi của đề tài.

Ngoài ra, hệ thống được giới hạn ở mô hình hai lõi sử dụng chung bộ nhớ và phối hợp thông qua cơ chế phân xử truy cập. Đề tài chưa triển khai bộ nhớ đệm cache, cơ chế đảm bảo tính nhất quán dữ liệu giữa các cache, hệ điều hành đa lõi, ngắt, ngoại lệ và các cơ chế quản lý bộ nhớ ảo. Việc đánh giá hệ thống chủ yếu được thực hiện thông qua mô phỏng và triển khai thử nghiệm trên kit FPGA KV260.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Định dạng lệnh

RISC-V là một kiến trúc tập lệnh mở, được thiết kế theo nguyên lý RISC với đặc điểm lệnh có cấu trúc đơn giản, độ dài lệnh cố định 32bit và dễ dàng mở rộng bằng các tập lệnh mở rộng. Trong đề tài này, nhóm thực hiện tập trung thiết kế bộ xử lý có khả năng thực thi tập lệnh RV32IMFA, trong đó RV32I là tập lệnh số nguyên cơ bản 32 bit, còn các phần mở rộng M, F và A lần lượt hỗ trợ các phép nhân/chia số nguyên, xử lý số thực dấu phẩy động đơn chính xác và các lệnh nguyên tử phục vụ truy cập dữ liệu chia sẻ.

Về cơ bản, các lệnh trong kiến trúc RISC-V được chia thành nhiều nhóm định dạng khác nhau, tùy theo chức năng và số lượng toán hạng của lệnh. Các định dạng lệnh chính bao gồm như trong Bảng 2.1.

Bảng 2.1: Định dạng lệnh của RISC-V

Định dạng lệnh						Nhóm lệnh
funct7	rs2	rs1	funct3	rd	opcode	R-type
Imm [11:0]		rs1	funct3	rd	opcode	I-type
Imm [11:5]	rs2	rs1	funct3	Imm [4:0]	opcode	S-type
Imm [12 10:5]	rs2	rs1	funct3	Imm [4:1 11]	opcode	B-type
Imm [31:12]				rd	opcode	U-type
Imm [20 10:1 11 19:12]				rd	opcode	J-type
rs3/fmt	rs2	rs1	rm	rd	opcode	R4-type
funct4			rd/rs1	rs2	opcode	CR-Type
funct3		imm	rd/rs1	imm	opcode	CI-Type

funct3		Uimme [5:2 7:6]		rs2	opcode	CSS-Type
funct3		Uzuimme [5:4 9:6 2 3]		rd'	opcode	CIW-Type
funct3	Uimme [5:3]	rs1'	Uimme [2 6]	rd'	opcode	CL-Type
funct3	Uimme [5:3]	rs1'/rd'	Uimme [2 6]	rs2'	opcode	CS-Type
funct6		rs1'/rd'	funct2	rs2'	opcode	CA-Type
funct3		offset	rs1'/rd'	offset	opcode	CB-Type
funct3		Imme [11 4 9:8 10 6 7 3:1 5]			opcode	CJ-Type

Dựa trên trường mã thao tác 7 bit (opcode), các định dạng lệnh trong kiến trúc RV32IMFA được phân biệt như sau:

- Opcode của R-type là 7b'0110011.
- Opcode của I-type là 7b'0110011, 7b'0000011.
- Opcode của S-type là 7b'0100011.
- Opcode của B-type là 7b'1100011.
- Opcode của U-type là 7b'0110111, 7b'0010111.
- Opcode của J-type là 7b'1101111.
- Opcode của R4-type là 7b'1000011 đối với lệnh MADD, 7'b1000111 với lệnh MSUB, 7'b1001111 đối với lệnh NMADD, 7'b1001011 đối với lệnh NMSUB.

2.2. Chức năng lệnh

Chức năng của từng lệnh trong quá trình thực thi được mô tả chi tiết trong Bảng 2.2, giúp làm rõ vai trò và hoạt động của các lệnh trong kiến trúc tập lệnh được nghiên cứu.

Bảng 2.2: Cấu trúc và chức năng của từng lệnh RISC-V

Lệnh	Chức năng	Mô tả
add	$R[rd] = R[rs1] + R[rs2]$	Cộng hai giá trị thanh ghi rs1 và rs2 trong khối RF, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
and	$R[rd] = R[rs1] \& R[rs2]$	AND hai giá trị thanh ghi rs1 và rs2 trong khối RF, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
or	$R[rd] = R[rs1] R[rs2]$	OR hai giá trị thanh ghi rs1 và rs2 trong khối RF, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
xor	$R[rd] = R[rs1] \wedge R[rs2]$	XOR hai giá trị thanh ghi rs1 và rs2 trong khối RF, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
sll	$R[rd] = R[rs1] \ll R[rs2]$	Shift giá trị thanh ghi rs1 trong RF sang trái n bit, với n là giá trị của thanh ghi rs2, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
srl	$R[rd] = R[rs1] \gg R[rs2]$	Shift giá trị thanh ghi rs1 trong RF sang phải n bit, với n là giá trị của thanh ghi rs2, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
slt	$R[rd] = (R[rs1] < R[rs2]) ? 1 : 0$	So sánh 2 giá trị có dấu. Nếu giá trị thanh ghi rs1 nhỏ hơn giá trị thanh ghi rs2, kết quả sẽ lưu giá trị 1 vào thanh ghi rd. Ngược lại, kết quả sẽ lưu giá trị 0 vào thanh ghi rd.
sltu	$R[rd] = (R[rs1] < R[rs2]) ? 1 : 0$	So sánh 2 giá trị không dấu. Nếu giá trị thanh ghi rs1 nhỏ hơn giá trị thanh ghi rs2, kết quả sẽ lưu giá trị 1 vào thanh ghi rd. Ngược lại, kết quả sẽ lưu giá trị 0 vào thanh ghi rd.
sub	$R[rd] = R[rs1] - R[rs2]$	Trừ hai giá trị thanh ghi rs1 và rs2 trong khối RF, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.

sra	$R[rd] = R[rs1] \gg \gg R[rs2]$	Shift giá trị thanh ghi rs1 trong RF sang phải n bit, với n là giá trị của thanh ghi rs2. Trong quá trình thực thi, bit MSB của rs1 khi dịch phải sẽ lấy bit MSB của thanh ghi rs2. Kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
addi	$R[rd] = R[rs1] + imm$	Cộng hai giá trị thanh ghi rs1 và giá trị tức thời imm, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
andi	$R[rd] = R[rs1] \& imm$	AND hai giá trị thanh ghi rs1 và giá trị tức thời imm, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
ori	$R[rd] = R[rs1] imm$	OR hai giá trị thanh ghi rs1 và giá trị tức thời imm, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
xori	$R[rd] = R[rs1] \wedge imm$	XOR hai giá trị thanh ghi rs1 và giá trị tức thời imm, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
slti	$R[rd] = (R[rs1] < imm) ? 1 : 0$	So sánh 2 giá trị có dấu. Nếu giá trị thanh ghi rs1 nhỏ hơn giá trị tức thời imm, kết quả sẽ lưu giá trị 1 vào thanh ghi rd. Ngược lại, kết quả sẽ lưu giá trị 0 vào thanh ghi rd.
sltiu	$R[rd] = (R[rs1] < imm) ? 1 : 0$	So sánh 2 giá trị không dấu. Nếu giá trị thanh ghi rs1 nhỏ hơn giá trị tức thời imm, kết quả sẽ lưu giá trị 1 vào thanh ghi rd. Ngược lại, kết quả sẽ lưu giá trị 0 vào thanh ghi rd.
slli	$R[rd] = R[rs1] \ll imm$	Shift giá trị thanh ghi rs1 trong RF sang trái n bit, với n là giá trị tức thời imm, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
srli	$R[rd] = R[rs1] \gg imm$	Shift giá trị thanh ghi rs1 trong RF sang phải n bit, với n là giá trị tức thời imm, kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.

srai	$R[rd] = R[rs1] \gg \gg \text{imm}$	Shift giá trị thanh ghi rs1 trong RF sang phải n bit, với n là giá trị tức thời imm. Trong quá trình thực thi, bit MSB của thanh ghi rs1 khi dịch phải sẽ lấy bit MSB của giá trị tức thời imm. Kết quả tính toán sẽ được ghi vào thanh ghi rd trong khối RF.
lb	$R[rd] = \{24'bM[(7), M[R[rs1] + \text{imm}](7:0)]$	Bộ xử lý sẽ trở tới thanh ghi địa chỉ $R[rs1] + \text{imm}$ trong khối Data Memory. Sau đó, bộ xử lý sẽ lấy 8 bit LSB giá trị của thanh ghi địa chỉ được trở đến vào thanh ghi rd và mở rộng 24 bit còn lại dựa vào bit 7 của giá trị được lấy.
lh	$R[rd] = \{16'bM[(15), M[R[rs1] + \text{imm}](15:0)]$	Bộ xử lý sẽ trở tới thanh ghi địa chỉ $R[rs1] + \text{imm}$ trong khối Data Memory. Sau đó, bộ xử lý sẽ lấy 16 bit LSB giá trị của thanh ghi địa chỉ được trở đến vào thanh ghi rd và mở rộng 16 bit còn lại dựa vào bit 15 của giá trị được lấy.
lw	$R[rd] = \{32'bM[R[rs1] + \text{imm}](31:0)]$	Bộ xử lý sẽ trở tới thanh ghi địa chỉ $R[rs1] + \text{imm}$ trong khối Data Memory. Sau đó, bộ xử lý sẽ lấy giá trị thanh ghi địa chỉ được trở đến vào thanh ghi rd.
lbu	$R[rd] = \{24'b0, M[R[rs1] + \text{imm}](7:0)]$	Bộ xử lý sẽ trở tới thanh ghi địa chỉ $R[rs1] + \text{imm}$ trong khối Data Memory. Sau đó, bộ xử lý sẽ lấy 8 bit LSB giá trị của thanh ghi địa chỉ được trở đến vào thanh ghi rd và mở rộng 24 bit còn lại bằng 0.
lhu	$R[rd] = \{16'b0, M[R[rs1] + \text{imm}](7:0)]$	Bộ xử lý sẽ trở tới thanh ghi địa chỉ $R[rs1] + \text{imm}$ trong khối Data Memory. Sau đó, bộ xử lý sẽ lấy 16 bit LSB giá trị của thanh ghi địa chỉ được trở đến vào thanh ghi rd và mở rộng 16 bit còn lại bằng 0.
jalr	$R[rd] = PC + 4; PC = R[rs1] + \text{imm}$	Kết quả tính toán $PC + 4$ sẽ được ghi vào thanh ghi rd của RF. Địa chỉ lệnh thực thi kế tiếp sẽ là kết quả cộng của thanh ghi rs1 và giá trị tức thời imm.
sb	$M[R[s1] + \text{imm}](7:0) = R[rs2](7:0)$	Bộ xử lý sẽ trở tới thanh ghi địa chỉ $R[rs1] + \text{imm}$ trong khối Data Memory. Sau đó, bộ xử lý sẽ ghi 8bit LSB

		giá trị thanh ghi rs2 vào thanh ghi địa chỉ được trỏ đến và mở rộng 24bit còn lại dựa vào bit 7 của giá trị thanh ghi rs2.
sh	$M[R[s1]+imm] (15:0) = R[rs2](15:0)$	Bộ xử lý sẽ trỏ tới thanh ghi địa chỉ $R[rs1] + imm$ trong khối Data Memory. Sau đó, bộ xử lý sẽ ghi 16bit LSB giá trị thanh ghi rs2 vào thanh ghi địa chỉ được trỏ đến và mở rộng 16bit còn lại dựa vào bit 15 của giá trị thanh ghi rs2.
sw	$M[R[s1] + imm] (31:0) = R[rs2] (31:0)$	Bộ xử lý sẽ trỏ tới thanh ghi địa chỉ $R[rs1] + imm$ trong khối Data Memory. Sau đó, bộ xử lý sẽ ghi giá trị thanh ghi rs2 vào thanh ghi địa chỉ được trỏ đến.
beq	If($R[rs1] == R[rs2]$) PC = PC + {imm,1'b0}	Nếu giá trị thanh ghi rs1 và rs2 bằng nhau. Địa chỉ lệnh thực thi kế tiếp sẽ là kết quả cộng của địa chỉ lệnh hiện tại và kết quả shift trái 1bit của giá trị tức thời.
bne	If($R[rs1] != R[rs2]$) PC = PC + {imm,1'b0}	Nếu giá trị thanh ghi rs1 và rs2 khác nhau. Địa chỉ lệnh thực thi kế tiếp sẽ là kết quả cộng của địa chỉ lệnh hiện tại và kết quả shift trái 1bit của giá trị tức thời.
blt	If($R[rs1] < R[rs2]$) PC = PC + {imm,1'b0}	Nếu giá trị thanh ghi rs1 nhỏ hơn giá trị thanh ghi rs2. Địa chỉ lệnh thực thi kế tiếp sẽ là kết quả cộng của địa chỉ lệnh hiện tại và kết quả shift trái 1bit của giá trị tức thời.
bge	If($R[rs1] >= R[rs2]$) PC = PC + {imm,1'b0}	Nếu giá trị thanh ghi rs1 lớn hơn hoặc bằng giá trị thanh ghi rs2. Địa chỉ lệnh thực thi kế tiếp sẽ là kết quả cộng của địa chỉ lệnh hiện tại và kết quả shift trái 1bit của giá trị tức thời.
bltu	If($R[rs1] < R[rs2]$) PC = PC + {imm,1'b0}	Kiểm tra điều kiện hai giá trị không dấu. Nếu giá trị thanh ghi rs1 nhỏ hơn giá trị thanh ghi rs2. Địa chỉ lệnh thực thi kế tiếp sẽ là kết quả cộng của địa chỉ lệnh hiện tại và kết quả shift trái 1bit của giá trị tức thời.
bgeu	If($R[rs1] >= R[rs2]$) PC = PC + {imm,1'b0}	Kiểm tra điều kiện hai giá trị không dấu. Nếu giá trị thanh ghi rs1 lớn hơn hoặc bằng giá trị thanh ghi rs2.

		Địa chỉ lệnh thực thi kế tiếp sẽ là kết quả cộng của địa chỉ lệnh hiện tại và kết quả shift trái 1bit của giá trị tức thời.
jal	$R[rd] = PC + 4; PC = PC + \{imm, 1'b0\}$	Kết quả tính toán $PC + 4$ sẽ được ghi vào thanh ghi rd của RF. Địa chỉ lệnh thực thi kế tiếp sẽ là kết quả cộng của địa chỉ lệnh hiện tại và kết quả shift trái 1bit của giá trị tức thời.
auipc	$R[rd] = PC + \{imm, 12'b0\}$	Kết quả cộng của địa chỉ lệnh hiện tại và kết quả shift trái 12 bit của giá trị tức thời sẽ được ghi vào thanh ghi rd của RF.
lui	$R[rd] = \{imm, 12'b0\}$	Kết quả shift trái 12bit của giá trị tức thời sẽ được ghi vào thanh ghi rd của RF.
FMADD.S	$fd = fs1 + fs2$	Lấy nội dung thanh ghi fs1 và fs2 (float). Tính tổng dấu chấm động và ghi kết quả vào fd
FMSUB.S	$fd = (fs1 * fs2) - fs3$	Nhân và trừ hợp nhất hai số float, giảm sai số làm tròn.
FNMSUB.S	$fd = -((fs1 * fs2) - fs3)$	Thực hiện nhân và trừ hợp nhất rồi đổi dấu kết quả.
C.JR	$PC = R[rs1]$	Nhảy tới địa chỉ chứa trong thanh ghi RS1
C.JALR	$R[1] = PC + 2; PC = R[rs1]$	Nhảy tới địa chỉ trong RS1 và lưu địa chỉ trở về
C.LWSP	$R[rd] = MEM[R[2] + offset]$	Load 32-bit word từ bộ nhớ lệch từ SP
C.FLWSP	$F[rd] = MEM[R[2] + offset]$	Load số thực 32-bit từ địa chỉ lệch
C.LI	$R[rd] = imm$	Nạp hằng số vào thanh ghi.
C.LUI	$R[rd] = imm \ll 12$	Nạp hằng số vào phần trên của thanh ghi (Upper Immediate).
C.ADDI	$R[rd] = R[rd] + imm$	Cộng giá trị tức thời vào thanh ghi.
C.ADDI16SP	$R[2] = R[2] + imm$	Điều chỉnh SP bằng một giá trị nhỏ (tăng/giảm SP)
C.SLLI	$R[rd] = R[rd] \ll shamt$	Dịch trái logic.

C.NOP	Không thực hiện gì	Không thực hiện gì
C.SWSP	$\text{MEM}[\text{R}[2] + \text{offset}] = \text{R}[\text{rs2}]$	Load số thực 32-bit từ địa chỉ lệch
C.FSWSP	$\text{MEM}[\text{R}[2] + \text{offset}] = \text{F}[\text{rs2}]$	Store số thực vào bộ nhớ lệch từ
C.ADDI4SPN	$\text{R}[\text{rd}'] = \text{R}[2] + \text{imm}$	Tính địa chỉ lệch từ SP, dùng cho load/store nhanh.
C.LW	$\text{R}[\text{rd}'] = \text{MEM}[\text{R}[\text{rs1}'] + \text{offset}]$	Load 32-bit word từ địa chỉ bộ nhớ.
C.FLW	$\text{F}[\text{rd}'] = \text{MEM}[\text{R}[\text{rs1}'] + \text{offset}]$	Load số thực 32-bit từ bộ nhớ.
C.SW	$\text{MEM}[\text{R}[\text{rs1}'] + \text{offset}] = \text{R}[\text{rs2}']$	Store word vào địa chỉ bộ nhớ.
C.FSW	$\text{MEM}[\text{R}[\text{rs1}'] + \text{offset}] = \text{F}[\text{rs2}']$	Store float vào bộ nhớ.
C.AND	$\text{R}[\text{rd}] = \text{R}[\text{rd}] \& \text{R}[\text{rs2}]$	AND bit giữa hai thanh ghi.
C.OR	$\text{R}[\text{rd}] = \text{R}[\text{rd}] \mid \text{R}[\text{rs2}]$	OR bit giữa hai thanh ghi.
C.XOR	$\text{R}[\text{rd}] = \text{R}[\text{rd}] \oplus \text{R}[\text{rs2}]$	XOR bit giữa hai thanh ghi.
C.SUB	$\text{R}[\text{rd}] = \text{R}[\text{rd}] - \text{R}[\text{rs2}]$	Trừ hai thanh ghi.
C.BEQZ	if ($\text{R}[\text{rs1}] == 0$) then PC = PC + offset	Nhảy có điều kiện nếu thanh ghi bằng 0.
C.BNEZ	if ($\text{R}[\text{rs1}] \neq 0$) then PC = PC + offset	Nhảy có điều kiện nếu thanh ghi khác 0.
C.SRLI	$\text{R}[\text{rd}] = \text{R}[\text{rd}] \gg \text{shamt}$ (logic)	Dịch phải logic (điền 0 vào bit trái).
C.SRAI	$\text{R}[\text{rd}] = \text{R}[\text{rd}] \gg \text{shamt}$ (arith)	Dịch phải số học (bảo toàn bit dấu).
C.ANDI	$\text{R}[\text{rd}] = \text{R}[\text{rd}] \& \text{imm}$	AND giữa thanh ghi và hằng số.
C.J	PC = PC + offset	Nhảy không điều kiện tới địa chỉ đích lệch từ PC.
C.JAL	$\text{R}[1] = \text{PC} + 2$ PC = PC + offset	Nhảy không điều kiện, lưu địa chỉ trở về vào thanh ghi RA

2.3. Kiến trúc đa lõi

Kiến trúc đa lõi là kiến trúc trong đó một hệ thống xử lý tích hợp từ hai lõi xử lý trở lên, cho phép nhiều luồng lệnh hoặc nhiều chương trình được thực thi song song. Thay vì chỉ tăng tần số hoạt động của một lõi đơn, kiến trúc đa lõi tập trung khai thác khả năng xử lý đồng thời nhằm nâng cao hiệu năng tổng thể của hệ thống. Trong các hệ thống nhúng và hệ thống trên chip hiện đại, kiến trúc đa lõi được sử dụng rộng rãi để đáp ứng nhu cầu tính toán ngày càng cao, đồng thời vẫn đảm bảo khả năng mở rộng và tối ưu tài nguyên phần cứng.

Trong một hệ thống đa lõi, mỗi lõi xử lý có thể được thiết kế như một đơn vị thực thi độc lập, bao gồm các khối chức năng như bộ nạp lệnh, bộ giải mã, ALU, tập thanh ghi, khối truy cập bộ nhớ và các cơ chế điều khiển pipeline. Các lõi có thể thực thi các chương trình khác nhau hoặc cùng phối hợp xử lý một bài toán chung. Tuy nhiên, khi nhiều lõi cùng hoạt động trong một hệ thống, vấn đề quan trọng không chỉ nằm ở khả năng thực thi lệnh của từng lõi, mà còn ở cách các lõi chia sẻ tài nguyên và trao đổi dữ liệu với nhau.

Một trong những tài nguyên thường được chia sẻ trong hệ thống đa lõi là bộ nhớ dữ liệu. Khi hai hoặc nhiều lõi cùng truy cập vào bộ nhớ dùng chung, có thể xảy ra tình huống nhiều yêu cầu đọc hoặc ghi xuất hiện tại cùng một thời điểm. Nếu không có cơ chế điều phối phù hợp, các yêu cầu này có thể gây xung đột trên bus dữ liệu, làm sai lệch dữ liệu hoặc khiến hệ thống hoạt động không ổn định. Vì vậy, hệ thống đa lõi cần có khối phân xử truy cập bộ nhớ để quyết định lõi nào được quyền sử dụng bus tại một thời điểm nhất định.

Khối phân xử có nhiệm vụ tiếp nhận các yêu cầu truy cập bộ nhớ từ các lõi, lựa chọn một yêu cầu hợp lệ để phục vụ trước, đồng thời giữ các yêu cầu còn lại ở trạng thái chờ. Cơ chế này giúp đảm bảo tại một thời điểm chỉ có một lõi được phép điều khiển bus truy cập bộ nhớ, từ đó tránh hiện tượng tranh chấp tài nguyên. Tùy theo yêu cầu thiết kế, khối phân xử có thể sử dụng nhiều chính sách khác nhau như ưu tiên cố định hoặc luân phiên. Trong đó, cơ chế luân phiên thường được sử dụng để đảm bảo tính công bằng giữa các lõi khi nhiều yêu cầu truy cập xuất hiện liên tục.

Bên cạnh vấn đề phân xử truy cập, đồng bộ dữ liệu cũng là một nội dung quan trọng trong hệ thống đa lõi. Khi nhiều lõi cùng thao tác trên một vùng nhớ chung, đặc biệt là trong các thao tác đọc-sửa-ghi, dữ liệu có thể bị sai nếu quá trình truy cập của một lõi bị xen ngang bởi lõi khác. Ví dụ, hai lõi cùng đọc một giá trị trong bộ nhớ, cùng tính toán và cùng ghi kết quả trở lại có thể làm mất một lần cập nhật dữ liệu. Hiện tượng này thường được gọi là tranh chấp dữ liệu và là một trong những vấn đề cốt lõi cần xử lý trong môi trường đa lõi.

Để hỗ trợ đồng bộ dữ liệu, kiến trúc RISC-V cung cấp mở rộng A, bao gồm các lệnh nguyên tử như LR.W, SC.W và các lệnh AMO. Các lệnh này cho phép thực hiện các thao tác trên bộ nhớ theo cách không bị chia cắt, giúp đảm bảo tính đúng đắn khi nhiều lõi cùng truy cập vùng nhớ dùng chung. Cặp lệnh LR.W và SC.W được sử dụng để thực hiện cơ chế load-reserved và store-conditional, trong đó lệnh ghi chỉ thành công nếu vùng nhớ được giữ chỗ không bị lõi khác thay đổi. Các lệnh AMO như AMOADD.W, AMOSWAP.W, AMOAND.W, AMOOR.W và AMOXOR.W cho phép thực hiện thao tác đọc-sửa-ghi trên bộ nhớ một cách nguyên tử.

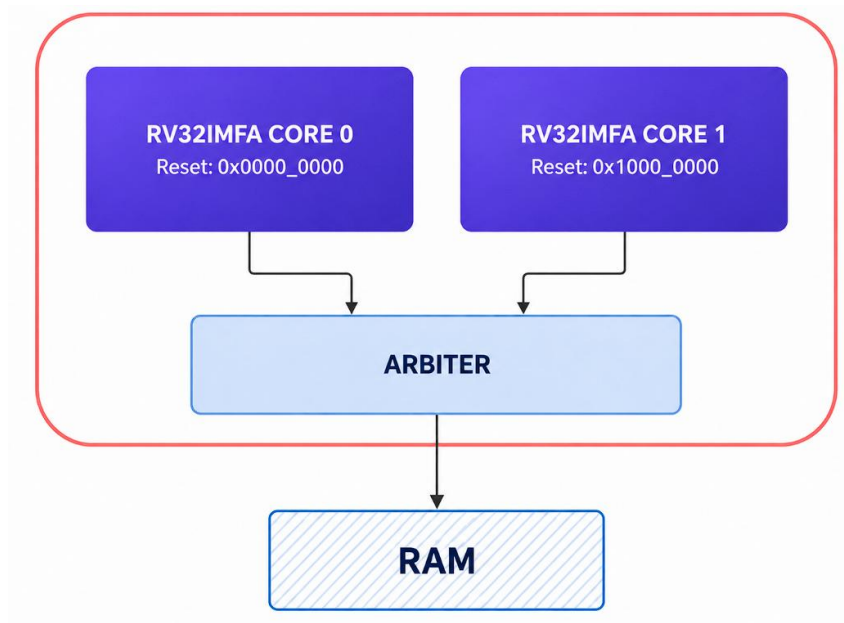
Trong phạm vi đề tài, kiến trúc đa lõi được hiện thực dưới dạng hệ thống hai lõi RV32IMFA. Mỗi lõi có khả năng thực thi tập lệnh RISC-V 32-bit với các mở rộng I, M, F và A. Hai lõi hoạt động song song và cùng truy cập một vùng bộ nhớ dữ liệu dùng chung thông qua khối phân xử. Trọng tâm của thiết kế là đảm bảo hai lõi có thể phát sinh yêu cầu truy cập bộ nhớ, được phân xử đúng thứ tự và nhận dữ liệu phản hồi chính xác. Đồng thời, các lệnh nguyên tử được sử dụng để hỗ trợ các thao tác đồng bộ trong trường hợp hai lõi cùng thao tác trên cùng một vùng nhớ.

Việc nghiên cứu và thiết kế kiến trúc đa lõi trong đề tài có ý nghĩa quan trọng vì nó mở rộng phạm vi của bộ xử lý từ một lõi đơn sang một hệ thống có khả năng xử lý song song. Đây là nền tảng để đánh giá các vấn đề thực tế trong thiết kế vi xử lý hiện đại như tranh chấp tài nguyên, phân xử bus, đồng bộ bộ nhớ và kiểm thử hoạt động giữa nhiều lõi. Qua đó, hệ thống hai lõi RV32IMFA không chỉ kiểm chứng khả năng thực thi tập lệnh của từng lõi, mà còn làm rõ khả năng phối hợp giữa các lõi trong môi trường bộ nhớ dùng chung.

Chương 3. THIẾT KẾ HỆ THỐNG

3.1. Thiết kế hệ thống tổng quát

Để hiện thực bộ vi xử lý dựa trên kiến trúc tập lệnh RISC-V, đề tài tập trung thiết kế phần cứng của hệ thống xử lý, bao gồm hai lõi RV32IMFA, khối phân xử truy cập bộ nhớ Arbiter và bộ nhớ dùng chung. Cấu trúc tổng quát của hệ thống được trình bày trong Hình 3.1, trong đó hai lõi xử lý cùng truy cập bộ nhớ thông qua khối Arbiter.



Hình 3.1: Sơ đồ tổng quát hệ thống dual-core RV32IMFA

Từ sơ đồ tổng quát ở Hình 3.1, có thể thấy hệ thống được cấu thành từ các khối chính gồm hai lõi xử lý RV32IMFA, khối Arbiter và bộ nhớ dùng chung. Chức năng cụ thể của từng khối được trình bày trong Bảng 3.1.

Bảng 3.1: Chức năng các khối trong hệ thống dual-core RV32IMFA

Tên	Mô tả
Core 0	Là lõi xử lý thứ nhất trong hệ thống, được thiết kế theo kiến trúc tập lệnh RISC-V 32-bit và hỗ trợ các mở rộng I, M, F và A. Core 0 bắt đầu thực thi chương trình từ địa chỉ reset 0x0000_0000.
Core 1	Là lõi xử lý thứ hai trong hệ thống, có kiến trúc tương tự Core 0. Core 1 bắt đầu thực thi chương trình từ địa chỉ reset 0x1000_0000, cho phép hai

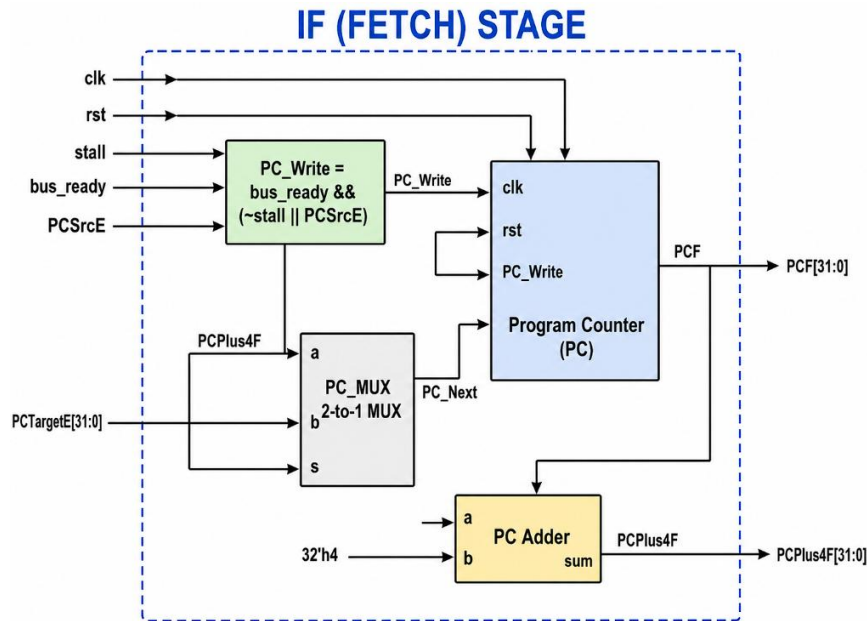
	lỗi chạy các vùng chương trình khác nhau trong bộ nhớ lệnh.
Arbiter	Là khối phân xử truy cập bộ nhớ giữa hai lõi xử lý. Khi Core 0 và Core 1 cùng phát sinh yêu cầu truy cập RAM, Arbiter sẽ lựa chọn một lõi được quyền thực hiện giao dịch tại một thời điểm, nhằm tránh xung đột trên bus dữ liệu.
RAM	Là vùng bộ nhớ dùng chung trong quá trình mô phỏng hệ thống. Nội dung bộ nhớ được khởi tạo từ file mã máy (.mem) bằng lệnh \$readmemh. RAM được sử dụng để cung cấp dữ liệu/chương trình cho hệ thống kiểm thử, không phải là khối thiết kế RTL trọng tâm của đề tài.
Bus kết nối	Là các đường tín hiệu dùng để truyền địa chỉ, dữ liệu và tín hiệu điều khiển giữa các core, Arbiter và RAM. Bus giúp liên kết các thành phần trong hệ thống thành một kiến trúc xử lý hoàn chỉnh.

3.2. Thiết kế chi tiết

3.2.1. Thiết kế phân cứng

a. Tầng IF

Thành phần chính của tầng IF là thanh ghi bộ đếm chương trình (Program Counter - PC), có nhiệm vụ lưu trữ địa chỉ của lệnh đang được tìm nạp trong từng chu kỳ xung nhịp. Giá trị PCF từ thanh ghi PC được đưa đến bộ nhớ lệnh (Instruction Memory - IMEM) để đọc ra lệnh tương ứng tại địa chỉ đó. Đồng thời, khối cộng địa chỉ sẽ tạo giá trị PCPlus4F bằng cách cộng thêm 4 vào địa chỉ hiện tại nhằm chuẩn bị cho việc thực thi tuần tự các lệnh. Bộ chọn kênh (MUX) tại đầu vào của thanh ghi PC có nhiệm vụ lựa chọn giá trị PC_Next giữa địa chỉ kế tiếp theo tuần tự (PCPlus4F) và địa chỉ đích của các lệnh rẽ nhánh hoặc nhảy (PC_SrcE), tùy thuộc vào tín hiệu điều khiển được tạo ra từ các tầng phía sau. Cơ chế này giúp bộ xử lý có thể thực hiện cả quá trình tìm nạp tuần tự lẫn thay đổi luồng thực thi khi gặp các lệnh điều khiển chương trình. Sau khi hoàn tất quá trình tìm nạp, các tín hiệu được tạo ra tại tầng IF, bao gồm PCF, PCPlus4F và InstrF, sẽ được lưu vào thanh ghi pipeline IF/ID để truyền sang tầng giải mã lệnh (ID) ở chu kỳ tiếp theo. Kiến trúc tổng thể của tầng IF được minh họa trong Hình 3.2.



Hình 3.2: Sơ đồ chi tiết tầng IF của bộ xử lý RV32IMFA

Để làm rõ chức năng của các đường tín hiệu trong tầng này, Bảng 3.2 trình bày các tín hiệu I/O chính của tầng IF.

Bảng 3.2: Các tín hiệu I/O của tầng IF

Tên tín hiệu	Độ rộng	I/O	Mô tả
clk, rst	1 bit	In	Đồng hồ và Reset.
stall	1 bit	In	Tín hiệu dừng từ Hazard Unit.
bus_ready	1 bit	In	Tín hiệu báo Bus bộ nhớ sẵn sàng.
PCSrcE	1 bit	In	Chọn nguồn PC (0: PC+4, 1: PCTarget).
PCTargetE	32 bit	In	Địa chỉ nhảy (từ EX).
imem_instr	32 bit	In	Lệnh đọc từ IMEM.
imem_addr	32 bit	Out	Địa chỉ gửi tới IMEM.
InstrF	32 bit	Out	Lệnh gửi sang ID.
PCF, PCPlus4F	32 bit	Out	PC và PC+4 gửi sang ID.

b. Tầng ID

Tầng ID có nhiệm vụ giải mã lệnh và chuẩn bị dữ liệu cần thiết cho các tầng xử lý phía sau. Lệnh 32-bit nhận từ tầng IF được phân tách thành các trường như opcode, funct3, funct7, rs1, rs2, rs3, rd và trường immediate. Các trường này được

đưa đến các khối chức năng bên trong tầng ID để tạo tín hiệu điều khiển, đọc dữ liệu từ tập thanh ghi và tạo giá trị tức thời.

Khối Control_Unit nhận các trường opcode, funct3, funct7 và rs2 từ lệnh đầu vào để phân loại loại lệnh đang thực thi. Từ đó, khối này tạo ra các tín hiệu điều khiển như RegWriteD, FPCRegWriteD, ALUSrcD, MemWriteD, MemReadD, ResultSrcD, BranchD, JumpD, ALUControlD, FPUControlD, MemOpD, CSR_D, Fence_D và AtomicD. Các tín hiệu này sẽ được truyền sang các tầng sau để điều khiển quá trình thực thi lệnh.

Khối Register_File là tập thanh ghi số nguyên, được sử dụng cho các lệnh thuộc nhóm RV32I, RV32M và RV32A. Khối này đọc hai toán hạng nguồn tại các địa chỉ rs1 và rs2, tương ứng với các trường InstrD[19:15] và InstrD[24:20]. Ngoài ra, dữ liệu kết quả từ tầng WB được ghi ngược về tập thanh ghi thông qua tín hiệu RegWriteW, địa chỉ ghi RD_W và dữ liệu ghi ResultW.

Khối FP_Register_File là tập thanh ghi dấu phẩy động, được sử dụng cho các lệnh thuộc mở rộng F và nhóm lệnh FMA. Khối này đọc các toán hạng dấu phẩy động fs1, fs2 và fs3 từ các trường tương ứng trong lệnh. Việc ghi kết quả dấu phẩy động từ tầng WB được điều khiển bởi tín hiệu FPCRegWriteW, địa chỉ ghi RD_F_W và dữ liệu ghi ResultW.

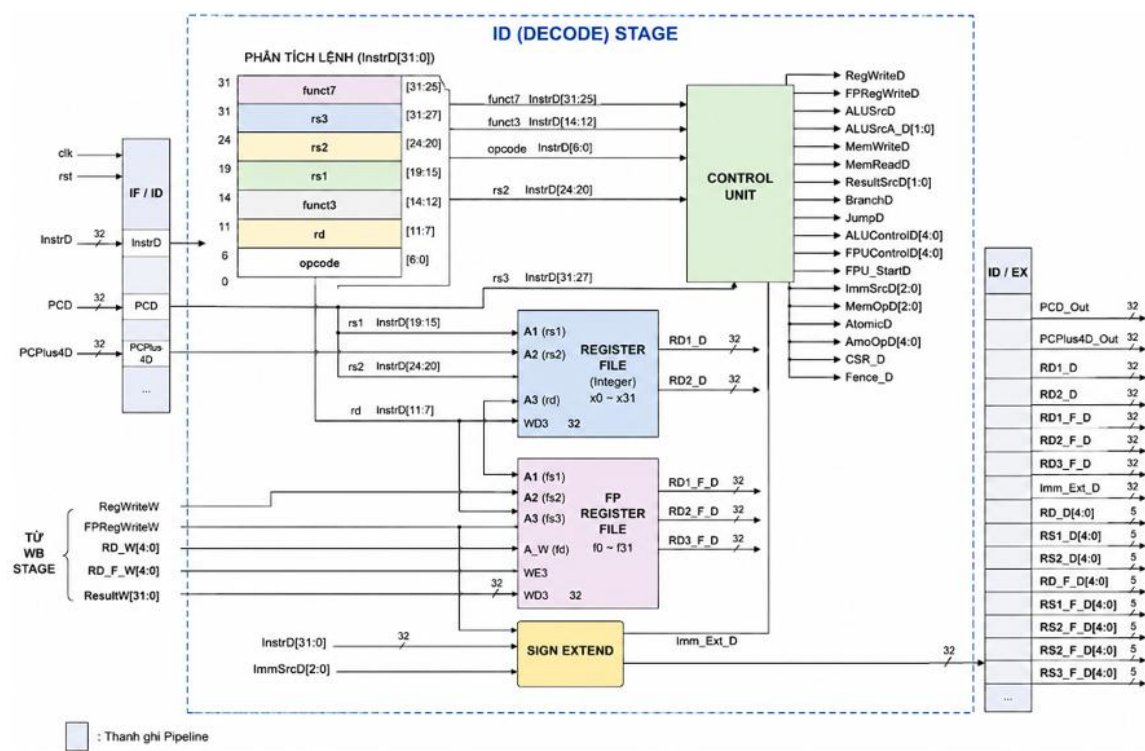
Khối Sign_Extend có nhiệm vụ tạo giá trị tức thời Imm_Ext_D từ lệnh đầu vào InstrD. Tùy theo loại định dạng lệnh được xác định bởi tín hiệu ImmSrcD, các bit immediate sẽ được sắp xếp lại và mở rộng dấu để tạo thành dữ liệu 32-bit sử dụng ở các tầng sau. Các tín hiệu I/O chính của tầng này được mô tả trong Bảng 3.3.

Bảng 3.3: Các trường của tập lệnh RISC-V 32-bit

Tên	Vị trí
opcode	i_Inst[6:0]
funct3	i_Inst[14:12]
funct7	i_Inst[31:25]
rs1	i_Inst[19:15]
rs2	i_Inst[24:20]

rs3	i_Inst[31:27]
rd	i_Inst[11:7]

Từ các trường được tách ra trong mã lệnh 32-bit, tầng ID sử dụng thông tin này để điều khiển các khối chức năng bên trong như Control_Unit, Register_File, FP_Register_File và Sign_Extend. Các khối này phối hợp để tạo tín hiệu điều khiển, đọc toán hạng từ tập thanh ghi và sinh giá trị tức thời phục vụ cho các tầng xử lý phía sau. Kiến trúc tổng quát của tầng ID được trình bày trong Hình 3.3.



Hình 3.3: Sơ đồ chi tiết tầng ID của bộ xử lý RV32IMFA

Để làm rõ vai trò của các đường tín hiệu trong tầng này, các tín hiệu vào/ra chính của module decode_stage được tổng hợp trong Bảng 3.4.

Bảng 3.4: Các tín hiệu I/O của tầng ID

Tên tín hiệu	Độ rộng	I/O	Mô tả
clk, rst	1 bit	Input	Clock và Reset.
PCD	32 bit	Input	PC của lệnh hiện tại.
InstrD	32 bit	Input	Mã lệnh 32-bit được truyền từ tầng IF sang tầng ID

PCD	32 bit	Input	Giá trị PC của lệnh hiện tại tại tầng ID
PCPlus4D	32 bit	Input	Giá trị PC + 4 của lệnh hiện tại, được truyền từ tầng IF.
RegWriteW	1 bit	Input	Tín hiệu cho phép ghi vào tập thanh ghi số nguyên, được truyền từ tầng WB.
FRegWriteW	1 bit	Input	Tín hiệu cho phép ghi vào tập thanh ghi dấu phẩy động, được truyền từ tầng WB.
RD_W	5 bit	Input	Địa chỉ thanh ghi đích của tập thanh ghi số nguyên tại tầng WB.
RD_F_W	5 bit	Input	Địa chỉ thanh ghi đích của tập thanh ghi dấu phẩy động tại tầng WB.
ResultW	32 bit	Input	Dữ liệu kết quả từ tầng WB dùng để ghi vào tập thanh ghi.
RegWriteD	1 bit	Output	Tín hiệu điều khiển cho phép ghi tập thanh ghi số nguyên.
FRegWriteD	2 bit	Output	Tín hiệu điều khiển cho phép ghi tập thanh ghi dấu phẩy động
ALUSrcD	1 bit	Output	Tín hiệu chọn nguồn toán hạng thứ hai cho ALU.
ALUSrcA_D	2 bit	Output	Tín hiệu chọn nguồn toán hạng thứ nhất cho ALU
MemWriteD	1 bit	Output	Tín hiệu điều khiển ghi dữ liệu vào bộ nhớ.
MemReadD	1 bit	Output	Tín hiệu điều khiển đọc dữ liệu từ bộ nhớ.
ResultSrcD	2 bit	Output	Tín hiệu chọn nguồn dữ liệu ghi về thanh ghi ở tầng WB
BranchD	1 bit	Output	Tín hiệu xác định lệnh hiện tại là lệnh rẽ nhánh có điều kiện.
JumpD	1 bit	Output	Tín hiệu xác định lệnh hiện tại là lệnh nhảy
ALUControlD	5 bit	Output	Mã điều khiển phép toán cho khối ALU.
FPUControlD	5 bit	Output	Mã điều khiển phép toán cho khối FPU.
FPU_StartD	1 bit	Output	Tín hiệu khởi động FPU đối với các lệnh tính toán dấu phẩy động và nhóm lệnh FMA.
ImmSrcD	3 bit	Output	Tín hiệu chọn kiểu mở rộng immediate trong khối

			Sign_Extend.
MemOpD	3 bit	Output	Tín hiệu xác định kiểu truy cập bộ nhớ, ví dụ load/store byte, half-word hoặc word.
AtomicD	1 bit	Output	Tín hiệu xác định lệnh thuộc nhóm atomic RV32A.
AmoOpD	5 bit	Output	Mã thao tác AMO, được lấy từ trường InstrD[31:27].
RD1_D, RD2_D	32 bit	Output	Dữ liệu đọc từ thanh ghi nguồn rs1 và rs2 của tập thanh ghi số nguyên.
RD1_F_D, RD1_F_D, RD3_F_D	32 bit	Output	Dữ liệu đọc từ thanh ghi nguồn fs1, fs2 của tập thanh ghi dấu phẩy động.
Imm_Ext_D	32 bit	Output	Giá trị immediate sau khi được sắp xếp bit và mở rộng dấu.
PCD_Out	32 bit	Output	Giá trị PCD được truyền tiếp sang tầng sau.
PCPlus4D_Out	32 bit	Output	Giá trị PCPlus4D được truyền tiếp sang tầng sau.
RD_D, RS1_D, RS2_D	32 bit	Output	Địa chỉ thanh ghi đích rd của tập thanh ghi số nguyên
RD_F_D, RS1_F_D, RS2_F_D, RS3_F_D	32 bit	Output	Địa chỉ thanh ghi nguồn fs1, fs2, fs3, fd

Các tín hiệu đầu vào và đầu ra của tập thanh ghi số nguyên được mô tả như Bảng 3.5.

Bảng 3.5: Các tín hiệu vào/ra của khối Register File

Tên tín hiệu	Độ rộng	I/O	Mô tả
WE3	1 bit	Input	Write Enable (Cho phép ghi).
A1, A2	5 bit	Input	Địa chỉ đọc 1 (Rs1) và 2 (Rs2).
A3	5 bit	Input	Địa chỉ ghi (Rd).
WD3	32 bit	Input	Dữ liệu cần ghi (Write Data).

RD1, RD2	32 bit	Output	Dữ liệu đọc ra 1 và 2.
----------	--------	--------	------------------------

Khối FP Register File có nhiệm vụ lưu trữ các toán hạng và kết quả của các phép toán dấu chấm động. Các tín hiệu đầu vào và đầu ra của khối được trình bày trong Bảng 3.6, bao gồm các tín hiệu địa chỉ đọc, địa chỉ ghi, dữ liệu ghi và dữ liệu đọc.

Bảng 3.6: Các tín hiệu vào/ra của khối FP_Register File

Tên tín hiệu	Độ rộng	I/O	Mô tả
WE3	1 bit	Input	Write Enable.
A1, A2, A3	5 bit	Input	Địa chỉ đọc Rs1, Rs2, Rs3 (cho FMA).
A_W	5 bit	Input	Địa chỉ ghi Rd.
WD3	32 bit	Input	Dữ liệu cần ghi.
RD1, RD2, RD3	32 bit	Output	Dữ liệu đọc ra.

Cơ chế lựa chọn dữ liệu ghi vào tập thanh ghi được điều khiển thông qua các tín hiệu điều khiển từ các tầng trong pipeline. Điều kiện hoạt động tương ứng với từng giá trị điều khiển của khối được mô tả trong Bảng 3.7.

Bảng 3.7: Điều kiện hoạt động của khối FP_Register File

Giá trị	Hoạt động
2'b00	Chọn kết quả của thanh ghi
2'b10	Chọn kết quả từ tầng MEM
2'b11	Chọn kết quả từ tầng WB

Khối Control Unit có nhiệm vụ giải mã các trường lệnh như opcode, funct3, funct7 và rs2 để tạo ra các tín hiệu điều khiển cho các tầng xử lý phía sau. Trong thiết kế này, Control Unit được xây dựng từ các khối con gồm Main Decoder, ALU Decoder và FPU Decoder, nhằm điều khiển quá trình thực thi của các nhóm lệnh số nguyên, truy cập bộ nhớ, nhảy/rẽ nhánh, nguyên tử và dấu chấm động. Các tín hiệu đầu vào và đầu ra của khối được trình bày trong Bảng 3.8.

Bảng 3.8: Các tín hiệu I/O của khối Control Unit

Tên tín hiệu	Độ rộng	I/O	Mô tả
Op	7 bit	Input	Opcode của lệnh.
funct3, funct7	5 bit	Input	Các trường chức năng của lệnh.
RegWrite, ALUSrc...	1 bit	Output	Các cờ điều khiển chính.
ALUControl, FPUControl	5 bit	Output	Mã điều khiển cho ALU và FPU.

Dựa trên kết quả giải mã lệnh, khối Control Unit tạo ra các tín hiệu điều khiển để lựa chọn nguồn dữ liệu và điều khiển quá trình ghi vào tập thanh ghi. Điều kiện tạo các tín hiệu ghi vào tập thanh ghi số nguyên và tập thanh ghi dấu chấm động trong từng trường hợp thực thi được trình bày trong Bảng 3.9.

Bảng 3.9: Điều kiện tạo tín hiệu ghi thanh ghi trong Control Unit

Tín hiệu	Điều kiện kích hoạt	Mô tả
RegWrite	RegWrite_Main = 1	Ghi kết quả về tập thanh ghi số nguyên đối với các lệnh số nguyên, load, jump hoặc atomic.
RegWrite	Lệnh FPU hợp lệ và thuộc nhóm ghi về thanh ghi số nguyên	Áp dụng cho các lệnh như so sánh dấu phẩy động, chuyển đổi float sang integer, FMV.X.W, FCLASS.S.
FPURegWrite	Lệnh FLW	Ghi dữ liệu load từ bộ nhớ vào tập thanh ghi dấu phẩy động.
FPURegWrite	Lệnh FPU và không thuộc nhóm ghi về thanh ghi số nguyên	Ghi kết quả tính toán dấu phẩy động vào tập thanh ghi dấu phẩy động.

Khối Sign Extend có nhiệm vụ tạo giá trị hằng tức thời theo từng định dạng lệnh của kiến trúc RISC-V. Dựa trên loại lệnh được giải mã, khối sẽ trích xuất các trường bit tương ứng và thực hiện mở rộng dấu để tạo ra giá trị hằng 32 bit phục vụ cho các phép toán, tính toán địa chỉ hoặc điều khiển luồng chương trình. Các tín hiệu đầu vào và đầu ra của khối được trình bày trong Bảng 3.10.

Bảng 3.10: Các tín hiệu I/O của khối Sign Extend

Tên	Độ rộng dữ liệu	I/O	Mô tả
In	32	I	Lệnh đầu vào
ImmSrc	3	I	Tín hiệu điều khiển lựa chọn chế độ cho khối ImmGen
Imm_Ext	32	O	Hằng số 32-bit sau khi mở rộng dấu

Khối Sign Extend hỗ trợ nhiều chế độ hoạt động tương ứng với các định dạng lệnh khác nhau của kiến trúc RISC-V. Tùy theo giá trị của tín hiệu ImmSrc, khối sẽ lựa chọn phương thức trích xuất và mở rộng dấu phù hợp. Các chế độ hoạt động của khối được trình bày trong Bảng 3.11.

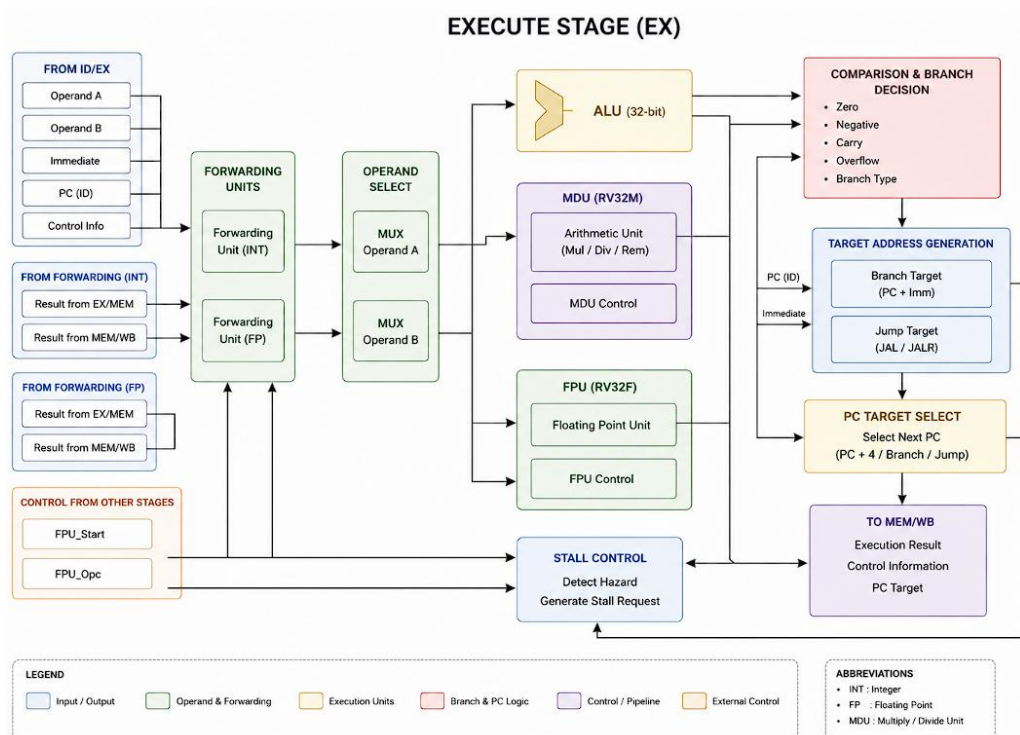
Bảng 3.11: Bảng mô tả hoạt động của khối Sign Extend

ImmSrc	Mô tả
3'b000	Định dạng I
3'b001	Định dạng S
3'b010	Định dạng B
3'b011	Định dạng U
3'b100	Định dạng J

c. Tầng EX

Đây là tầng thực hiện phần lớn các phép xử lý và tính toán của bộ vi xử lý. Tại tầng này, các toán hạng được nhận từ thanh ghi pipeline ID/EX và được đưa đến các khối chức năng tương ứng để thực hiện phép toán. Đối với các lệnh số nguyên, dữ liệu sẽ được xử lý bởi khối ALU; các lệnh nhân, chia và lấy dư được chuyển đến khối MDU; trong khi các lệnh dấu chấm động được thực hiện bởi khối FPU. Đồng thời, tầng EX cũng thực hiện việc đánh giá điều kiện rẽ nhánh, tính toán địa chỉ đích của các lệnh nhảy và tạo các tín hiệu điều khiển phục vụ cho việc cập nhật giá trị của thanh ghi PC. Kết quả tính toán cùng các tín hiệu điều khiển sau khi hoàn thành sẽ

được lưu vào thanh ghi pipeline EX/MEM để truyền sang tầng MEM ở chu kỳ tiếp theo. Kiến trúc tổng quát của tầng EX được trình bày trong Hình 3.4.



Hình 3.4: Sơ đồ khối tầng thực thi lệnh (EX) của bộ xử lý RV32IMFA

Dựa trên kiến trúc của tầng EX, các tín hiệu vào/ra của tầng này bao gồm dữ liệu toán hạng từ tầng ID, tín hiệu điều khiển thực thi, tín hiệu forwarding từ các tầng MEM/WB và các tín hiệu kết quả truyền sang tầng MEM. Các tín hiệu chính của module execute stage được mô tả trong Bảng 3.12.

Bảng 3.12: Các tín hiệu I/O của tầng EX

Tên tín hiệu	Độ rộng	I/O	Mô tả
clk, rst	1 bit	Input	Tín hiệu clock và reset của tầng EX.
RD1_E, RD2_E	32 bit	Input	Hai toán hạng số nguyên được truyền từ tầng ID sang tầng EX
PCE	32 bit	Input	Giá trị PC của lệnh hiện tại tại tầng EX.
PCPlus4E	32 bit	Input	Giá trị PC + 4 của lệnh hiện tại.
Imm_Ext_E	32 bit	Input	Giá trị immediate đã được mở rộng dấu từ tầng ID.
ALUControlE	5 bit	Input	Mã điều khiển phép toán cho ALU hoặc MDU.
ALUSrcE	32 bit	Input	Chọn toán hạng B của ALU giữa dữ liệu thanh ghi và

			immediate.
ALUSrcA_E	2 bit	Input	Chọn toán hạng A của ALU giữa dữ liệu thanh ghi, PC hoặc hằng số 0.
AtomicE	1 bit	Input	Tín hiệu xác định lệnh atomic tại tầng EX.
ResultSrcE	2 bit	Input	Chọn loại kết quả thực thi; trong thiết kế, 2'b11 dùng để xác định kết quả từ FPU.
MemWriteE	1 bit	Input	Tín hiệu điều khiển ghi bộ nhớ được truyền sang các tầng sau.
BranchE	1 bit	Input	Tín hiệu xác định lệnh rẽ nhánh có điều kiện.
JumpE	1 bit	Input	Tín hiệu xác định lệnh nhảy.
BranchTypeE	3 bit	Input	Mã loại rẽ nhánh, dùng để xác định các lệnh BEQ, BNE, BLT, BGE, BLTU, BGEU.
OpE	7 bit	Input	Opcode của lệnh, dùng để nhận biết một số lệnh đặc biệt như JALR hoặc FSW.
ForwardA_E, ForwardB_E	2 bit	Input	Tín hiệu chọn forwarding cho hai toán hạng số nguyên.
ResultW	32 bit	Input	Dữ liệu forwarding số nguyên từ tầng WB.
ALUResultM	32 bit	Input	Dữ liệu forwarding số nguyên từ tầng MEM
FPU_StartE	1 bit	Input	Tín hiệu khởi động FPU tại tầng EX.
FPU_Opcode_E	5 bit	Input	Mã điều khiển phép toán FPU.
RD1_F_E, RD2_F_E, RD3_F_E	32 bit	Input	Ba toán hạng dấu phẩy động được truyền từ tầng ID sang tầng EX.
ForwardA_F_E, ForwardB_F_E, ForwardC_F_E	2 bit	Input	Tín hiệu chọn forwarding cho ba toán hạng dấu phẩy động.
FP_ResultW	32 bit	Input	Dữ liệu forwarding dấu phẩy động từ tầng WB.
FP_ResultM	32 bit	Input	Dữ liệu forwarding dấu phẩy động từ tầng MEM.
Stall_FPU_Req	1 bit	Output	Yêu cầu dừng pipeline khi FPU đang thực thi và kết quả chưa sẵn sàng.
Stall_MDU_Req	1 bit	Output	Yêu cầu dừng pipeline khi MDU đang thực hiện phép

			nhân/chia và chưa hoàn tất.
ALUResultE	32 bit	Output	Kết quả cuối cùng của tầng EX, được chọn giữa kết quả ALU, MDU hoặc FPU.
WriteDataE	32 bit	Output	Dữ liệu dùng cho lệnh store. Với lệnh FSW, dữ liệu lấy từ thanh ghi dấu phẩy động; với store số nguyên, dữ liệu lấy từ thanh ghi số nguyên.
PCTargetE	32 bit	Output	Địa chỉ đích của lệnh nhảy hoặc rẽ nhánh. Với JALR, bit thấp nhất được ép về 0.
PCSrcE	1 bit	Output	Tín hiệu yêu cầu cập nhật PC khi có lệnh nhảy hoặc rẽ nhánh được thỏa mãn.
ZeroE	1 bit	Output	Cờ zero từ ALU, dùng cho xử lý rẽ nhánh.
NegativeE	1 bit	Output	Cờ âm từ ALU
CarryE	1 bit	Output	Cờ nhớ từ ALU.
OverFlowE	1 bit	Output	Cờ tràn số học từ ALU.

Khối ALU có nhiệm vụ thực hiện các phép toán số nguyên cơ bản thuộc tập lệnh RV32I và một số phép toán so sánh phục vụ các lệnh nguyên tử của mở rộng RV32A. Các tín hiệu đầu vào và đầu ra của khối ALU được trình bày trong Bảng 3.13.

Bảng 3.13: Các tín hiệu I/O của khối ALU

Tên tín hiệu	Độ rộng	I/O	Mô tả
A, B	32 bit	Input	Hai toán hạng đầu vào.
ALUControl	5 bit	Input	Mã chọn phép toán.
Result	32 bit	Output	Kết quả tính toán.
Zero	1 bit	Output	Cờ báo bằng 0.

Hoạt động của ALU được điều khiển thông qua các tín hiệu ALUOp, Funct3 và Funct7 do khối Control Unit tạo ra. Từ các tín hiệu này, ALU sẽ lựa chọn phép toán tương ứng và sinh ra mã điều khiển ALUSel để thực hiện phép tính phù hợp với từng loại lệnh. Mối quan hệ giữa các tín hiệu điều khiển và phép toán của ALU được trình bày trong Bảng 3.14.

Bảng 3.14: Mã điều khiển các phép toán trong ALU

Lệnh	ALUop	Funct7	Funct3	Hoạt động ALU	ALUSel
LOAD	2'b00	X	X	ADD	4'b0000
STORE	2'b00	X	X	ADD	4'b0000
AUIPC	2'b00	X	X	ADD	4'b0000
ADD	2'b01	7'b00000000	3'b000	ADD	4'b0000
SUB	2'b01	7'b01000000	3'b000	SUB	4'b0001
XOR	2'b01	7'b00000000	3'b100	XOR	4'b0010
OR	2'b01	7'b00000000	3'b110	OR	4'b0011
AND	2'b01	7'b00000000	3'b111	AND	4'b0100
SLL	2'b01	7'b00000000	3'b001	SLL	4'b0101
SRL	2'b01	7'b00000000	3'b101	SRL	4'b0110
SRA	2'b01	7'b01000000	3'b101	SRA	4'b0111
SLT	2'b01	7'b00000000	3'b010	SLT	4'b1000
SLTU	2'b01	7'b00000000	3'b011	SLTU	4'b1001
ADDI	2'b10	X	3'b000	ADD	4'b0000
XORI	2'b10	X	3'b100	XOR	4'b0010
ORI	2'b10	X	3'b110	OR	4'b0011
ANDI	2'b10	X	3'b111	AND	4'b0100
SLLI	2'b10	7'b00000000	3'b001	SLL	4'b0101
SRLI	2'b10	7'b00000000	3'b101	SRL	4'b0110
SRAI	2'b10	7'b01000000	3'b101	SRA	4'b0111
SLTI	2'b10	X	3'b010	SLT	4'b1000

Khối FPU có nhiệm vụ thực hiện các phép toán dấu chấm động đơn chính xác thuộc mở rộng RV32F, bao gồm các phép toán số học, so sánh, chuyển đổi kiểu dữ liệu, thao tác trên bit dấu và nhóm lệnh nhân-cộng kết. Tùy theo giá trị của tín hiệu `in_opcode`, khối FPU sẽ lựa chọn khối chức năng tương ứng để xử lý và trả kết quả thông qua tín hiệu `out_data` khi `out_valid` được kích hoạt. Các tín hiệu đầu vào và đầu ra của khối được trình bày trong Bảng 3.15.

Bảng 3.15: Các tín hiệu I/O trong FPU

Tên tín hiệu	Độ rộng	I/O	Mô tả
in_start	1 bit	Input	Kích hoạt FPU chạy.
in_opcode	5 bit	Input	Mã phép toán Float.
in_rs1, in_rs2	32 bit	Input	Toán hạng Float đầu vào.
out_data	32 bit	Output	Kết quả tính toán Float.
out_stall	1 bit	Output	Báo bận (khi đang chia/ngịch đảo).
out_valid	1 bit	Output	Tín hiệu báo kết quả FPU hợp lệ.

Mỗi phép toán dấu chấm động được xác định dựa trên các trường Opcode, Funct7, Funct3 và rs2 của lệnh. Dựa vào các trường này, khối FPU sẽ lựa chọn khối xử lý phù hợp để thực hiện phép toán tương ứng. Mối quan hệ giữa các trường điều khiển và chức năng của từng lệnh dấu chấm động được trình bày trong Bảng 3.16.

Bảng 3.16: Mã điều khiển các phép toán trong FPU

Lệnh	Opcode	Funct7	Funct3	rs2	Hoạt động FPU
FADD.S	1010011	0000000	X	X	Cộng số thực đơn chính xác
FSUB.S	1010011	0000100	X	X	Trừ số thực đơn chính xác
FMUL.S	1010011	0001000	X	X	Nhân số thực đơn chính xác
FDIV.S	1010011	0001100	X	X	Chia số thực đơn chính xác
FSQRT.S	1010011	0101100	X	00000	Căn bậc hai số thực
FSGNJ.S	1010011	0010000	000	X	Gán dấu từ rs2 cho rs1
FSGNJN.S	1010011	0010000	001	X	Gán dấu đảo của rs2 cho rs1
FSGNJX.S	1010011	0010000	010	X	Gán dấu bằng XOR dấu rs1 và rs2
FEQ.S	1010011	1010000	010	X	So sánh bằng
FLT.S	1010011	1010000	001	X	So sánh nhỏ hơn
FLE.S	1010011	1010000	000	X	So sánh nhỏ hơn hoặc bằng
FMIN.S	1010011	0010100	000	X	Chọn giá trị nhỏ hơn
FMAX.S	1010011	0010100	001	X	Chọn giá trị lớn hơn
FCVT.W.S	1010011	1100000	X	00000	Chuyển float sang integer có dấu
FCVT.WU.S	1010011	1100000	X	00001	Chuyển float sang integer

					không dấu
FCVT.S.W	1010011	1101000	X	00000	Chuyển integer có dấu sang float
FCVT.S.WU	1010011	1101000	X	00001	Chuyển integer không dấu sang float
FMV.X.W	1010011	1110000	000	00000	Chuyển nguyên bit từ FP sang integer
FMV.W.X	1010011	1111000	000	00000	Chuyển nguyên bit từ integer sang FP
FCLASS.S	1010011	1110000	001	00000	Phân loại giá trị float

Khối MDU có nhiệm vụ thực hiện các phép toán nhân, chia và lấy dư thuộc mở rộng RV32M. Trong thiết kế này, các phép toán nhân được thực hiện với độ trễ ngắn, trong khi các phép toán chia và lấy dư được xử lý theo từng chu kỳ bằng thuật toán chia lặp. Khối MDU nhận các toán hạng và mã điều khiển từ tầng thực thi, sau đó trả về kết quả sau khi quá trình tính toán hoàn tất. Các tín hiệu đầu vào và đầu ra của khối được trình bày trong Bảng 3.17.

Bảng 3.17: Các tín hiệu I/O của khối MDU

Tên tín hiệu	Độ rộng	I/O	Mô tả
start	1 bit	Input	Kích hoạt MDU bắt đầu thực hiện phép toán.
op	5 bit	Input	Mã phép toán nhân/chia/lấy dư thuộc nhóm RV32M.
A, B	32 bit	Input	Hai toán hạng đầu vào của khối MDU.
result	32 bit	Output	Kết quả tính toán của phép nhân, chia hoặc lấy dư.
busy	1 bit	Output	Báo MDU đang bận xử lý phép toán.
done	1 bit	Output	Tín hiệu báo kết quả MDU đã hợp lệ.

Khối MDU hỗ trợ các lệnh thuộc nhóm nhân, chia và lấy dư của tập lệnh RV32M. Mỗi phép toán được xác định thông qua tín hiệu điều khiển op, từ đó lựa chọn chức năng xử lý tương ứng và tạo ra kết quả phù hợp. Mã điều khiển và chức năng của từng phép toán được trình bày trong Bảng 3.18.

Bảng 3.18: Mã điều khiển các phép toán trong MDU

Lệnh	Opcode	Nhóm xử lý	Hoạt động MDU
MUL	5'b10000	Nhân	Nhân có dấu, lấy 32 bit thấp của kết quả.
MULH	5'b10001	Nhân	Nhân có dấu, lấy 32 bit cao của kết quả.
MULHSU	5'b10010	Nhân	Nhân A có dấu với B không dấu, lấy 32 bit cao.
MULHU	5'b10011	Nhân	Nhân không dấu, lấy 32 bit cao của kết quả.
DIV	5'b10100	Chia	Chia có dấu, trả về thương.
DIVU	5'b10101	Chia	Chia không dấu, trả về thương.
REM	5'b10110	Lấy dư	Chia có dấu, trả về phần dư.
REMU	5'b10111	Lấy dư	Chia không dấu, trả về phần dư.

d. Tầng MEM

Tầng MEM có nhiệm vụ thực hiện các thao tác truy cập bộ nhớ dữ liệu và giao tiếp với hệ thống bus thông qua khối phân xử (Arbiter). Địa chỉ truy cập bộ nhớ được lấy từ kết quả tính toán của tầng EX thông qua tín hiệu ALU_ResultM, trong khi dữ liệu ghi được cung cấp bởi tín hiệu WriteDataM. Dựa trên các tín hiệu điều khiển như MemReadM và MemWriteM, tầng MEM sẽ phát sinh các yêu cầu đọc hoặc ghi dữ liệu tới bộ nhớ. Đối với các lệnh đọc, dữ liệu sau khi được trả về từ bộ nhớ sẽ được chuyển đến tầng WB để ghi vào thanh ghi đích. Ngoài việc hỗ trợ các lệnh tải và lưu dữ liệu (load/store) thông thường, tầng MEM còn xử lý các lệnh nguyên tử thuộc mở rộng RV32A như LR.W, SC.W và các lệnh AMO. Các lệnh này sử dụng cơ chế reservation và giao tiếp với Arbiter nhằm đảm bảo tính nguyên tử khi nhiều lõi cùng truy cập vùng nhớ dùng chung. Kiến trúc tổng thể của tầng MEM được minh họa trong Hình 3.5.

			RV32A.
MemOpM	3 bit	Input	Xác định kiểu truy cập bộ nhớ như byte, half-word hoặc word.
ALU_ResultM	32 bit	Input	Địa chỉ truy cập bộ nhớ, được truyền từ kết quả tính toán ở tầng EX.
WriteDataM	32 bit	Input	Dữ liệu cần ghi vào bộ nhớ đối với lệnh store hoặc atomic.
Snoop_Addr	32 bit	Input	Địa chỉ ghi từ core khác, dùng để kiểm tra và hủy reservation của LR/SC.
Snoop_WE	1 bit	Input	Tín hiệu báo core khác đang ghi vào bộ nhớ.
bus_read_data	32 bit	Input	Dữ liệu đọc về từ bus hoặc bộ nhớ dùng chung.
bus_addr	32 bit	Output	Địa chỉ gửi ra bus, được gán bằng ALU_ResultM.
bus_write_data	32 bit	Output	Dữ liệu ghi ra bus. Với AMO, dữ liệu này là kết quả đọc-sửa-ghi.
bus_mem_write	1 bit	Output	Tín hiệu yêu cầu ghi bộ nhớ ra bus.
bus_mem_read	1 bit	Output	Tín hiệu yêu cầu đọc bộ nhớ ra bus.
bus_mem_op	3 bit	Output	Kiểu truy cập bộ nhớ gửi ra bus, được gán bằng MemOpM.
ReadDataM	32 bit	Output	Dữ liệu truyền sang tầng WB. Với SC.W, tín hiệu này trả về trạng thái thành công hoặc thất bại.

Bên cạnh các thao tác load/store thông thường, tầng MEM còn tích hợp cơ chế xử lý lệnh atomic RV32A. Cách hoạt động của tầng MEM trong từng trường hợp được trình bày trong Bảng 3.20.

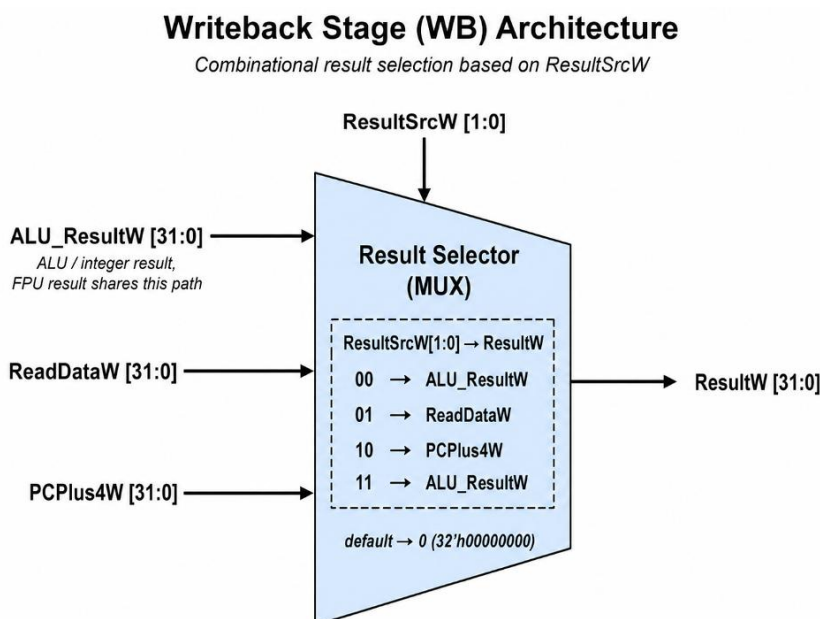
Bảng 3.20: Cách hoạt động của tầng MEM

Trường hợp	Điều kiện	Hoạt động
Load thông thường	AtomicM = 0, MemReadM = 1	Tầng MEM gửi yêu cầu đọc bộ nhớ qua bus_mem_read. Địa chỉ đọc là ALU_ResultM, dữ liệu đọc về từ bus_read_data được đưa ra ReadDataM.
Store thông	AtomicM = 0,	Tầng MEM gửi yêu cầu ghi bộ nhớ qua

thường	MemWriteM = 1	bus_mem_write. Địa chỉ ghi là ALU_ResultM, dữ liệu ghi là WriteDataM.
LR.W	AtomicM = 1, AmoOpM = AMO_LR	Tầng MEM đọc dữ liệu cũ từ bộ nhớ, đồng thời lưu ALU_ResultM vào reservation_addr và bật reservation_valid.
SC.W thành công	AtomicM = 1, AmoOpM = AMO_SC, reservation_valid = 1 và ALU_ResultM = reservation_addr	Tầng MEM ghi WriteDataM vào bộ nhớ và trả về ReadDataM = 0, biểu thị store-conditional thành công. Sau đó reservation bị xóa.
SC.W thất bại	AtomicM = 1, AmoOpM = AMO_SC, reservation không hợp lệ hoặc địa chỉ không khớp	Tầng MEM không ghi dữ liệu vào bộ nhớ và trả về ReadDataM = 1, biểu thị store-conditional thất bại.
AMO read-modify-write	AtomicM = 1, AmoOpM khác LR và SC	Tầng MEM đọc giá trị cũ từ bộ nhớ bus_read_data, tính dữ liệu mới amo_wdata dựa trên phép toán AMO và WriteDataM, sau đó ghi kết quả mới về bộ nhớ.
Hủy reservation	reservation_valid = 1, Snoop_WE = 1, Snoop_Addr = reservation_addr	Nếu core khác ghi vào đúng địa chỉ đang được reservation, tầng MEM xóa reservation_valid. Điều này làm cho lệnh SC.W sau đó bị thất bại.
Dữ liệu trả về WB với load/LR/AMO	Không phải SC.W	ReadDataM nhận giá trị bus_read_data, tức dữ liệu cũ đọc từ bộ nhớ.
Dữ liệu trả về WB với SC.W	is_SC = 1	ReadDataM = 0 nếu SC.W thành công, và ReadDataM = 1 nếu thất bại.

e. Tầng WB

Tầng WB là tầng cuối cùng trong pipeline, có nhiệm vụ lựa chọn dữ liệu kết quả để ghi ngược về tập thanh ghi. Dữ liệu ghi về có thể đến từ kết quả tính toán của ALU/FPU, dữ liệu đọc từ bộ nhớ hoặc giá trị PCPlus4W đối với các lệnh nhảy như JAL và JALR. Việc lựa chọn nguồn dữ liệu được điều khiển bởi tín hiệu ResultSrcW. Kiến trúc tổng quát của tầng WB được trình bày trong Hình 3.6.



Hình 3.6: Sơ đồ khối tầng WB của bộ xử lý RV32IMFA

Từ Hình 3.6 có thể thấy tầng WB hoạt động như một bộ chọn dữ liệu kết quả. Các ngõ vào của tầng này gồm kết quả từ ALU/FPU, dữ liệu đọc từ bộ nhớ và giá trị PCPlus4W. Dựa vào tín hiệu điều khiển ResultSrcW, tầng WB chọn một trong các nguồn dữ liệu này để tạo ra ResultW. Các tín hiệu vào/ra chính của tầng WB được mô tả trong Bảng 3.21.

Bảng 3.21: Các tín hiệu I/O của tầng WB

Tên tín hiệu	Độ rộng	I/O	Mô tả
ResultSrcW	2 bit	Input	Tín hiệu điều khiển lựa chọn nguồn dữ liệu ghi về thanh ghi.
ALU_ResultW	32 bit	Input	Kết quả từ ALU hoặc FPU được truyền đến tầng WB.

ReadDataW	32 bit	Input	Dữ liệu đọc từ bộ nhớ, dùng cho lệnh load, LR.W, hoặc trạng thái của SC.W.
PCPlus4W	32 bit	Input	Giá trị PC + 4, dùng làm dữ liệu ghi về đối với các lệnh nhảy như JAL và JALR.
ResultW	32 bit	Output	Dữ liệu cuối cùng được chọn để ghi ngược về tập thanh ghi.

Do tầng WB chỉ thực hiện chức năng chọn dữ liệu ghi về nên cách hoạt động của tầng này phụ thuộc trực tiếp vào giá trị của tín hiệu ResultSrcW. Bảng 3.22 trình bày nguồn dữ liệu được chọn tương ứng với từng giá trị của ResultSrcW.

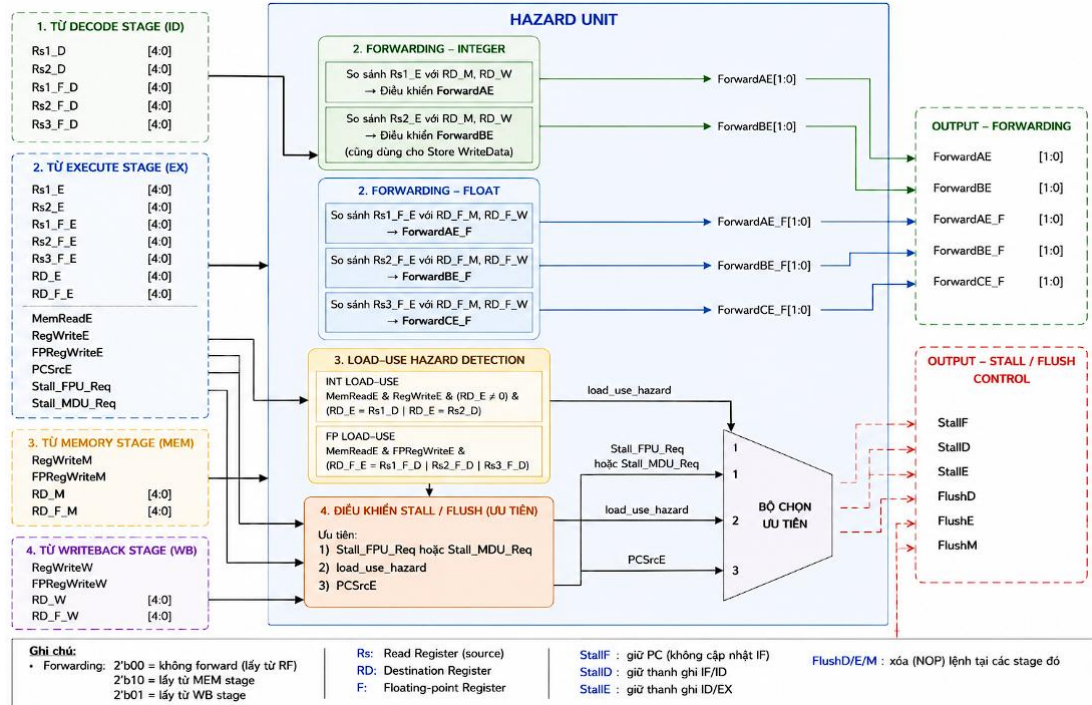
Bảng 3.22: Cách hoạt động của tầng WB

ResultSrcW	Nguồn dữ liệu được chọn	Giá trị gán cho ResultW	Mô tả
2'b00	Kết quả ALU/FPU	ALU_ResultW	Ghi kết quả tính toán từ ALU hoặc các lệnh xử lý thông thường về thanh ghi.
2'b01	Dữ liệu bộ nhớ	ReadDataW	Ghi dữ liệu đọc từ bộ nhớ về thanh ghi, dùng cho lệnh load, LR.W hoặc trạng thái SC.W.
2'b10	Giá trị PC + 4	PCPlus4W	Ghi địa chỉ quay về cho các lệnh nhảy như JAL và JALR.
2'b11	Kết quả FPU đi chung đường ALU	ALU_ResultW	Ghi kết quả FPU, do kết quả FPU đã được truyền chung qua đường ALU_ResultW.
Giá trị khác	Mặc định	32'h00000000	Tránh trạng thái không xác định khi tín hiệu điều khiển không hợp lệ.

f. Hazard Control

Khối Hazard Unit có nhiệm vụ phát hiện và xử lý các xung đột dữ liệu trong pipeline. Trong thiết kế này, Hazard Unit thực hiện hai chức năng tạo tín hiệu forwarding cho dữ liệu số nguyên và dấu phẩy động, đồng thời tạo các tín hiệu

stall/flush để điều khiển hoạt động của pipeline khi xảy ra load-use hazard, nhảy/rẽ nhánh hoặc khi các khối FPU/MDU đang xử lý lệnh nhiều chu kỳ. Kiến trúc tổng quát của khối Hazard Unit được trình bày trong Hình 3.7.



Hình 3.7: Sơ đồ khối Hazard Unit của bộ xử lý RV32IMFA

Dựa trên các thông tin này, khối tạo ra tín hiệu forwarding để chọn dữ liệu phù hợp, đồng thời phát sinh các tín hiệu stall/flush khi cần dừng hoặc xóa một số tầng trong pipeline. Các tín hiệu vào/ra chính của khối Hazard Unit được mô tả trong Bảng 3.23.

Bảng 3.23: Các tín hiệu I/O của khối Hazard Unit

Tên tín hiệu	Độ rộng	I/O	Mô tả
Rs1_D, Rs2_D	5 bit	Input	Địa chỉ hai thanh ghi nguồn số nguyên tại tầng ID.
Rs1_F_D, Rs2_F_D, Rs3_F_D	5 bit	Input	Địa chỉ các thanh ghi nguồn dấu phẩy động tại tầng ID.
Rs1_E, Rs2_E	5 bit	Input	Địa chỉ hai thanh ghi nguồn số nguyên tại tầng EX.
Rs1_F_E, Rs2_F_E, Rs3_F_E	5 bit	Input	Địa chỉ các thanh ghi nguồn dấu phẩy động tại tầng EX.
RD_E, RD_F_E	5 bit	Input	Địa chỉ thanh ghi đích số nguyên và dấu phẩy động

			tại tầng EX.
MemReadE	1 bit	Input	Tín hiệu cho biết lệnh tại tầng EX là lệnh đọc bộ nhớ.
RegWriteE	1 bit	Input	Tín hiệu cho phép ghi thanh ghi số nguyên tại tầng EX.
FRegWriteE	1 bit	Input	Tín hiệu cho phép ghi thanh ghi dấu phẩy động tại tầng EX.
PCSrcE	1 bit	Input	Tín hiệu cho biết có nhảy hoặc rẽ nhánh được thực hiện tại tầng EX.
Stall_FPU_Req	1 bit	Input	Yêu cầu dừng pipeline khi FPU đang xử lý lệnh nhiều chu kỳ.
Stall_MDU_Req	1 bit	Input	Yêu cầu dừng pipeline khi MDU đang xử lý lệnh nhân/chia nhiều chu kỳ.
RegWriteM, FRegWriteM	1 bit	Input	Tín hiệu cho phép ghi thanh ghi số nguyên và dấu phẩy động tại tầng MEM.
RD_M, RD_F_M	5 bit	Input	Địa chỉ thanh ghi đích số nguyên và dấu phẩy động tại tầng MEM.
RegWriteW, FRegWriteW	1 bit	Input	Tín hiệu cho phép ghi thanh ghi số nguyên và dấu phẩy động tại tầng WB.
RD_W, RD_F_W	5 bit	Input	Địa chỉ thanh ghi đích số nguyên và dấu phẩy động tại tầng WB.
ForwardAE, ForwardBE	2 bit	Output	Tín hiệu chọn forwarding cho hai toán hạng số nguyên tại tầng EX.
ForwardAE_F, ForwardBE_F, ForwardCE_F	2 bit	Output	Tín hiệu chọn forwarding cho các toán hạng dấu phẩy động tại tầng EX.
StallF, StallD, StallE	1 bit	Output	Tín hiệu dừng các tầng IF, ID và EX.
FlushD, FlushE, FlushM	1 bit	Output	Tín hiệu xóa lệnh tại các tầng ID, EX và MEM.

Cơ chế hoạt động của Hazard Unit được chia thành ba nhóm chính: forwarding dữ liệu, phát hiện load-use hazard và điều khiển stall/flush pipeline. Trong đó, các yêu cầu stall từ FPU/MDU có mức ưu tiên cao nhất, tiếp theo là load-use hazard và cuối cùng là xử lý nhảy/rẽ nhánh. Cách hoạt động của khối Hazard Unit được mô tả trong Bảng 3.24.

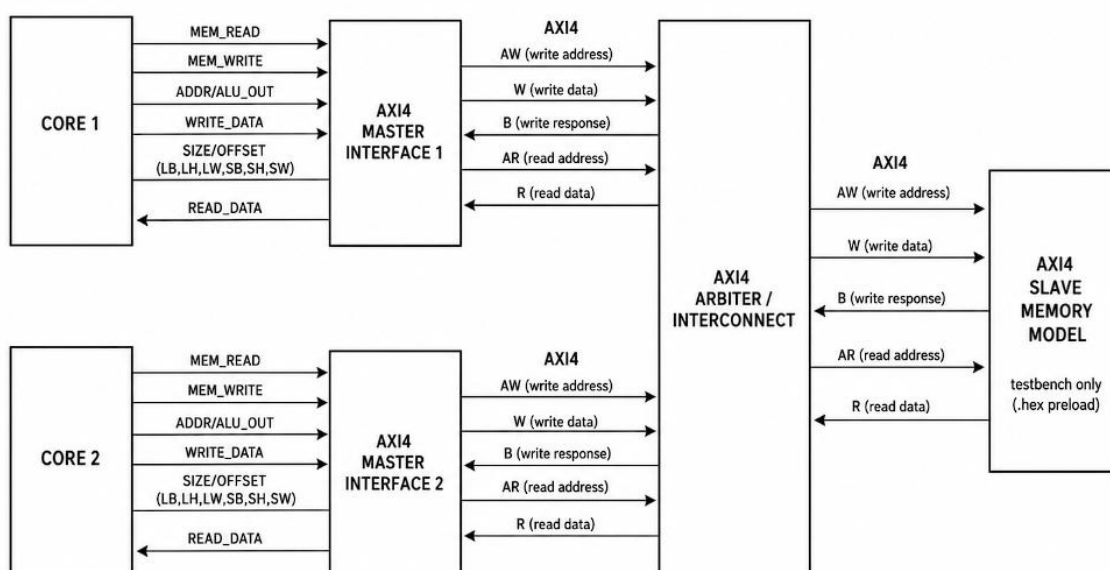
Bảng 3.24: Cách hoạt động của khối Hazard Unit

Trường hợp	Hoạt động
Forwarding số nguyên từ MEM	Tạo ForwardAE = 2'b10 hoặc ForwardBE = 2'b10 để lấy dữ liệu từ tầng MEM.
Forwarding số nguyên từ WB	Tạo ForwardAE = 2'b01 hoặc ForwardBE = 2'b01 để lấy dữ liệu từ tầng WB.
Không forwarding số nguyên	ForwardAE = 2'b00, ForwardBE = 2'b00, toán hạng lấy trực tiếp từ thanh ghi.
Forwarding dấu phẩy động từ MEM	Tạo ForwardAE_F, ForwardBE_F hoặc ForwardCE_F = 2'b10 để lấy dữ liệu dấu phẩy động từ tầng MEM.
Forwarding dấu phẩy động từ WB	Tạo ForwardAE_F, ForwardBE_F hoặc ForwardCE_F = 2'b01 để lấy dữ liệu dấu phẩy động từ tầng WB.
Load-use hazard số nguyên	Pipeline bị dừng tại IF và ID, đồng thời tầng EX bị flush để chèn bubble.
Load-use hazard dấu phẩy động	Pipeline bị dừng tại IF và ID, đồng thời tầng EX bị flush để chèn bubble.
FPU/MDU đang bận	StallF = 1, StallID = 1, StallE = 1, FlushM = 1. Pipeline giữ lệnh đang xử lý ở EX và không cho dữ liệu rác đi xuống MEM.
Load-use hazard	StallF = 1, StallID = 1, FlushE = 1. PC và IF/ID được giữ nguyên, ID/EX bị xóa.
Nhảy hoặc rẽ nhánh được chọn	FlushD = 1, FlushE = 1. Các lệnh sai đường trong pipeline bị xóa.
Không có hazard	Tất cả tín hiệu stall/flush giữ giá trị 0, pipeline hoạt động bình thường.

3.2.2. Thiết kế giao tiếp bộ nhớ theo chuẩn AXI4

Trong rong hệ thống RV32IMFA, khối giao tiếp bộ nhớ được xây dựng nhằm kết nối lõi xử lý với bộ nhớ lệnh và bộ nhớ dữ liệu bên ngoài. Thiết kế sử dụng các tín hiệu giao tiếp theo hướng AXI4, bao gồm các kênh địa chỉ đọc, dữ liệu đọc, địa chỉ ghi, dữ liệu ghi và phản hồi ghi. Tuy nhiên, trong phạm vi đề tài, giao tiếp AXI4 được đơn giản hóa để phục vụ mô phỏng và kiểm chứng chức năng của bộ xử lý, chưa hiện thực đầy đủ toàn bộ các đặc tả của chuẩn AXI4 như burst transaction hay các tín hiệu phản hồi mở rộng. Việc đơn giản hóa giao tiếp giúp giảm độ phức tạp của thiết kế, đồng thời vẫn đáp ứng đầy đủ yêu cầu kiểm thử các chức năng của bộ xử lý trong môi trường mô phỏng.

Bộ nhớ lệnh và bộ nhớ dữ liệu trong thiết kế không được xây dựng như một khối RTL độc lập, mà được mô hình hóa trong testbench và khởi tạo từ file HEX. Khối wrapper có nhiệm vụ chuyển đổi các tín hiệu truy cập bộ nhớ từ lõi xử lý, bao gồm địa chỉ, dữ liệu ghi, tín hiệu đọc/ghi và kiểu truy cập bộ nhớ, sang các nhóm tín hiệu giao tiếp dạng AXI4 đơn giản hóa. Nhờ đó, bộ xử lý có thể truy cập bộ nhớ theo một giao diện thống nhất, đồng thời thuận tiện cho việc mở rộng và tích hợp với các mô hình bộ nhớ hoặc hệ thống ngoại vi trong các nghiên cứu tiếp theo. Kiến trúc giao tiếp bộ nhớ của hệ thống được trình bày trong Hình 3.8.



Hình 3.8: Sơ đồ giao tiếp bộ nhớ dạng AXI4 của hệ thống RV32IMFA

Giao tiếp bộ nhớ trong thiết kế được xây dựng theo hướng AXI4 đơn giản hóa. Các kênh giao tiếp được chia thành năm nhóm chính gồm kênh địa chỉ đọc, dữ liệu đọc, địa chỉ ghi, dữ liệu ghi và phản hồi ghi. Một số tín hiệu cơ bản đã được hiện thực trực tiếp trong module RV32IMFA_IP_Wrapper, trong khi một số tín hiệu mở rộng của AXI4 như burst, ID hoặc response có thể được đặt giá trị mặc định trong phạm vi mô phỏng.

a. Kênh địa chỉ đọc AR

Kênh địa chỉ đọc AR được sử dụng để truyền thông tin địa chỉ từ master đến bộ nhớ hoặc slave khi hệ thống thực hiện thao tác đọc dữ liệu. Đối với bộ nhớ dữ liệu, địa chỉ đọc được tạo ra từ kết quả tính toán địa chỉ ở tầng MEM khi bộ xử lý thực hiện các lệnh load hoặc các thao tác đọc dữ liệu liên quan. Các tín hiệu chính của kênh địa chỉ đọc AR được trình bày trong Bảng 3.25.

Bảng 3.25: Các tín hiệu của kênh địa chỉ đọc AR

Tên tín hiệu	Độ rộng	Mô tả
ARID	4 bit	ID của giao dịch đọc.
ARADDR	32 bit	Địa chỉ đọc.
ARLEN	8 bit	Số beat trong burst.
ARSIZE	3 bit	Kích thước mỗi beat.
ARBURST	2 bit	Kiểu burst.
ARLOCK	1 bit	Tín hiệu khóa giao dịch.
ARCACHE	4 bit	Thuộc tính cache của giao dịch đọc.
ARPROT	3 bit	Thuộc tính bảo vệ truy cập. Trong mô phỏng có thể gán mặc định.
ARVALID	1 bit	Master báo địa chỉ đọc hợp lệ. IMEM luôn bằng 1, DMEM được điều khiển bởi Mem_ReadEnM.
ARREADY	32 bit	Dữ liệu cần ghi (Write Data).

b. Kênh dữ liệu đọc R

Kênh dữ liệu đọc R được sử dụng để truyền dữ liệu phản hồi từ bộ nhớ hoặc slave về master sau khi yêu cầu đọc trên kênh AR được chấp nhận. Trong thiết kế

này, kênh R đảm nhiệm việc trả về dữ liệu đọc từ bộ nhớ lệnh hoặc bộ nhớ dữ liệu cho lõi xử lý. Đối với bộ nhớ lệnh, dữ liệu trả về là mã lệnh được truy xuất theo địa chỉ PC. Đối với bộ nhớ dữ liệu, dữ liệu trả về là giá trị được đọc từ địa chỉ do tăng MEM tạo ra khi thực hiện các lệnh load hoặc các thao tác đọc liên quan. Ngoài dữ liệu đọc, kênh R còn bao gồm các tín hiệu phản hồi trạng thái và tín hiệu hợp lệ nhằm xác nhận quá trình truyền dữ liệu. Các tín hiệu chính của kênh dữ liệu đọc R được trình bày trong Bảng 3.26.

Bảng 3.26: Các tín hiệu giao tiếp của kênh dữ liệu đọc R

Tên tín hiệu	Độ rộng	Mô tả
RID	4 bit	ID phản hồi đọc, tương ứng với ARID. Thiết kế hiện tại không dùng nhiều giao dịch outstanding nên có thể bỏ qua.
RDATA	32 bit	Dữ liệu đọc trả về. Với IMEM là M_AXI_IMEM_RDATA, với DMEM là M_AXI_DMEM_RDATA.
RRESP	2 bit	Trạng thái phản hồi đọc, ví dụ OKAY hoặc lỗi. Trong mô phỏng đơn giản có thể xem mặc định là OKAY.
RLAST	1 bit	Báo beat cuối của burst. Do thiết kế đọc đơn beat nên có thể gán 1.
RVALID	1 bit	Slave hoặc testbench báo dữ liệu đọc hợp lệ. Nếu M_AXI_IMEM_RVALID = 0, wrapper đưa lệnh NOP vào core.
RREADY	1 bit	Master báo sẵn sàng nhận dữ liệu đọc.

c. Kênh địa chỉ ghi AW

Kênh địa chỉ ghi AW được sử dụng để truyền thông tin địa chỉ ghi từ master đến bộ nhớ hoặc slave khi hệ thống thực hiện thao tác ghi dữ liệu. Địa chỉ ghi được tạo ra từ kết quả tính toán địa chỉ ở tầng MEM, sau đó được đưa ra ngoài thông qua tín hiệu địa chỉ ghi của giao tiếp AXI4. Các tín hiệu chính của kênh địa chỉ ghi AW được trình bày trong Bảng 3.27.

Bảng 3.27: Các tín hiệu giao tiếp của kênh địa chỉ ghi AW

Tên tín hiệu	Độ rộng	Mô tả
AWID	4 bit	ID của giao dịch ghi.

AWADDR	32 bit	Địa chỉ ghi bộ nhớ dữ liệu.
AWLEN	8 bit	Số beat trong burst ghi.
AWSIZE	3 bit	Kích thước mỗi beat.
AWBURST	2 bit	Kiểu burst ghi.
AWCACHE	4 bit	Thuộc tính cache cho giao dịch ghi.
AWPROT	3 bit	Thuộc tính bảo vệ truy cập ghi.
AWVALID	1 bit	Master báo địa chỉ ghi hợp lệ.
AWREADY	1 bit	Slave báo sẵn sàng nhận địa chỉ ghi.

d. Kênh dữ liệu ghi W

Kênh dữ liệu ghi W được sử dụng để truyền dữ liệu ghi từ master đến bộ nhớ hoặc slave sau khi địa chỉ ghi đã được xác định thông qua kênh AW. Trong thiết kế này, kênh W đảm nhiệm việc đưa dữ liệu từ lõi xử lý ra bộ nhớ dữ liệu khi thực hiện các lệnh store hoặc các thao tác ghi phát sinh từ lệnh nguyên tử. Ngoài dữ liệu ghi, kênh W còn sử dụng tín hiệuWSTRB để xác định byte nào trong word 32-bit được ghi, nhờ đó hệ thống có thể hỗ trợ các kiểu ghi khác nhau như SB, SH và SW. Các tín hiệu chính của kênh dữ liệu ghi W được trình bày trong Bảng 3.28.

Bảng 3.28: Các tín hiệu giao tiếp của kênh dữ liệu ghi W

Tên tín hiệu	Độ rộng	Mô tả
WDATA	32 bit	Dữ liệu ghi ra bộ nhớ
WSTRB	4 bit	Byte strobe xác định byte nào được ghi. Tín hiệu này được tạo bởi store unit để hỗ trợ SB, SH, SW.
WLAST	1 bit	Báo beat cuối của burst ghi.
WVALID	1 bit	Master báo dữ liệu hợp lệ.
WREADY	1 bit	Slave báo sẵn sàng nhận dữ liệu ghi.

e. Kênh phản hồi ghi B

Kênh phản hồi ghi B được sử dụng để trả về trạng thái phản hồi từ bộ nhớ hoặc slave sau khi thao tác ghi dữ liệu hoàn tất. Trong giao tiếp AXI4, sau khi master gửi địa chỉ ghi qua kênh AW và dữ liệu ghi qua kênh W, slave sẽ phản hồi lại thông qua kênh B để xác nhận giao dịch ghi đã được xử lý. Trong thiết kế này, tín hiệu phản hồi

ghi giúp hệ thống xác định thời điểm thao tác store hoàn tất, từ đó cho phép pipeline tiếp tục thực thi các lệnh tiếp theo. Các tín hiệu chính của kênh phản hồi ghi B được trình bày trong Bảng 3.29.

Bảng 3.29: Các tín hiệu giao tiếp của kênh phản hồi ghi B

Tên tín hiệu	Độ rộng	Mô tả
BID	4 bit	ID phản hồi ghi, tương ứng với AWID.
BRESP	2 bit	Trạng thái phản hồi ghi, ví dụ OKAY hoặc lỗi.
BVALID	1 bit	Slave báo phản hồi ghi hợp lệ. Khi BVALID = 1, thao tác store được xem là hoàn tất.
BREADY	1 bit	Master báo sẵn sàng nhận phản hồi ghi.

f. Vai trò của giao tiếp AXI4

Giao tiếp bộ nhớ dạng AXI4 đóng vai trò trung gian giữa lõi xử lý RV32IMFA và mô hình bộ nhớ trong testbench. Khối giao tiếp này nhận các yêu cầu truy cập bộ nhớ từ core, bao gồm địa chỉ truy cập, dữ liệu ghi, tín hiệu đọc/ghi và kiểu truy cập bộ nhớ. Sau đó, các tín hiệu này được chuyển đổi thành các kênh giao tiếp tương ứng như kênh địa chỉ đọc, dữ liệu đọc, địa chỉ ghi, dữ liệu ghi và phản hồi ghi.

Đối với bộ nhớ lệnh, giao tiếp chỉ sử dụng kênh đọc. Giá trị PCF từ core được đưa ra ngoài thông qua tín hiệu địa chỉ đọc M_AXI_IMEM_ARADDR, sau đó mô hình bộ nhớ trả về mã lệnh thông qua tín hiệu M_AXI_IMEM_RDATA. Nếu dữ liệu lệnh hợp lệ, wrapper sẽ đưa lệnh này vào core để tiếp tục quá trình thực thi.

Đối với bộ nhớ dữ liệu, giao tiếp hỗ trợ cả thao tác đọc và ghi. Khi thực hiện lệnh load, địa chỉ đọc được gửi ra thông qua kênh AR, dữ liệu trả về từ testbench được đưa qua load_unit để xử lý theo kiểu truy cập byte, half-word hoặc word trước khi trả về core. Khi thực hiện lệnh store, dữ liệu ghi từ core được đưa qua store_unit để tạo dữ liệu ghi và tín hiệu byte-enable WSTRB phù hợp với từng kiểu store như SB, SH hoặc SW.

Ngoài ra, khối wrapper còn sử dụng cơ chế store_pending để giữ yêu cầu ghi cho đến khi nhận được phản hồi ghi từ bộ nhớ thông qua tín hiệu M_AXI_DMEM_BVALID. Trong thời gian chờ phản hồi, tín hiệu d_stall được kích

hoạt để tạm dừng core, nhằm đảm bảo thao tác store hoàn tất trước khi pipeline tiếp tục thực thi.

Trong phạm vi đề tài, bộ nhớ lệnh và bộ nhớ dữ liệu không được thiết kế như một khối RTL độc lập, mà được mô hình hóa trong testbench và khởi tạo từ file HEX. Vì vậy, giao tiếp bộ nhớ trong thiết kế được xem là giao tiếp AXI4 đơn giản hóa, phục vụ mục tiêu mô phỏng và kiểm chứng chức năng của bộ xử lý thay vì hiện thực đầy đủ toàn bộ chuẩn AXI4.

g. Sơ đồ hệ thống

Ở mức hệ thống, thiết kế bao gồm hai lõi xử lý RV32IMFA, hoạt động song song và dùng chung một vùng bộ nhớ dữ liệu. Mỗi lõi có giao diện bộ nhớ lệnh riêng, trong đó Core 0 nhận lệnh thông qua cặp tín hiệu PC0F – Instr0F, còn Core 1 nhận lệnh thông qua cặp tín hiệu PC1F – Instr1F. Cách tổ chức này cho phép hai lõi thực thi chương trình độc lập ở phía lệnh, đồng thời tránh xung đột truy cập trên đường lệnh.

Ở phía dữ liệu, cả hai lõi đều phát sinh các tín hiệu truy cập bộ nhớ gồm địa chỉ, dữ liệu ghi, tín hiệu đọc/ghi và chế độ truy cập bộ nhớ. Do chỉ có một bộ nhớ dữ liệu dùng chung, các yêu cầu từ hai lõi sẽ được đưa vào khối Arbiter để phân xử quyền truy cập. Arbiter có nhiệm vụ lựa chọn một lõi được phép truy cập bộ nhớ tại một thời điểm, đồng thời phát tín hiệu stall tới lõi còn lại nếu lõi đó đang có yêu cầu nhưng chưa được cấp quyền. Cơ chế này giúp đảm bảo tính đúng đắn khi truy cập tài nguyên dùng chung, đồng thời duy trì tính công bằng giữa hai lõi.

Sau khi arbiter chọn một yêu cầu hợp lệ, các tín hiệu truy cập sẽ được chuyển tới giao diện bộ nhớ dùng chung thông qua các cổng Shared_Mem_Addr, Shared_Mem_WriteData, Shared_Mem_WriteEn, Shared_Mem_ReadEn và Shared_MemOp. Dữ liệu đọc từ bộ nhớ sẽ được trả về thông qua tín hiệu Shared_Mem_ReadData, sau đó được định tuyến ngược lại tới lõi đang được cấp quyền truy cập. Như vậy, wrapper đóng vai trò là khối liên kết trung tâm giữa các lõi xử lý và tài nguyên bộ nhớ dữ liệu dùng chung.

Ngoài chức năng phân xử truy cập bộ nhớ, hệ thống còn hỗ trợ cơ chế snoop/write commit giữa hai lõi. Cụ thể, khi một lõi thực hiện ghi dữ liệu, địa chỉ ghi và tín hiệu xác nhận ghi sẽ được chuyển sang lõi còn lại thông qua các tín hiệu Snoop_Addr và Snoop_WE. Cơ chế này cho phép lõi còn lại nhận biết sự kiện ghi dữ liệu từ đối tác, từ đó hỗ trợ duy trì tính nhất quán khi cả hai lõi cùng hoạt động trên không gian bộ nhớ chia sẻ.

Bên cạnh các tín hiệu chức năng chính, hệ thống còn cung cấp các tín hiệu gỡ lỗi như Core0_ResultW, Core1_ResultW, Core0_ALU_ResultE_Debug, Core1_ALU_ResultE_Debug, cùng với các tín hiệu trạng thái phân xử Grant0_Debug, Grant1_Debug và Turn_Debug. Các tín hiệu này rất hữu ích trong quá trình mô phỏng, kiểm thử và đánh giá hoạt động của toàn bộ hệ thống dual-core.

Tóm lại, sơ đồ hệ thống thể hiện rõ mô hình dual-core RV32IMFA với bộ nhớ lệnh tách biệt, bộ nhớ dữ liệu dùng chung và khối arbiter phân xử truy cập, qua đó đáp ứng mục tiêu xây dựng một kiến trúc đa lõi đơn giản, dễ kiểm chứng và phù hợp cho quá trình nghiên cứu, mô phỏng và triển khai phần cứng.

Chương 4. XÁC MINH VÀ ĐÁNH GIÁ THIẾT KẾ

4.1. Mục tiêu kiểm thử

Mục tiêu của quá trình kiểm thử là đánh giá tính đúng đắn và mức độ hoàn thiện của bộ xử lý RV32IMFA sau khi thiết kế. Quá trình kiểm thử được thực hiện nhằm xác nhận rằng bộ xử lý có khả năng thực thi chính xác các nhóm lệnh thuộc kiến trúc RV32IMFA, bao gồm nhóm lệnh số nguyên cơ bản RV32I, nhóm lệnh nhân chia RV32M, nhóm lệnh dấu chấm động RV32F và nhóm lệnh nguyên tử RV32A.

Trước hết, việc kiểm thử tập trung vào từng nhóm lệnh độc lập để đảm bảo các khối chức năng bên trong bộ xử lý hoạt động đúng theo yêu cầu thiết kế. Đối với nhóm lệnh RV32I, các phép toán số học, logic, truy cập bộ nhớ, rẽ nhánh và nhảy được kiểm tra nhằm xác nhận hoạt động của ALU, bộ thanh ghi, bộ nhớ và mạch cập nhật PC. Đối với nhóm lệnh RV32M, quá trình kiểm thử tập trung vào các phép nhân, chia, lấy phần dư và các trường hợp đặc biệt như chia cho 0 hoặc tràn số có dấu. Với nhóm lệnh RV32F, mục tiêu là kiểm tra hoạt động của khối FPU, thanh ghi dấu chấm động, các phép tính số thực và các trường hợp đặc biệt theo chuẩn IEEE 754 như NaN hoặc Infinity. Đối với nhóm lệnh RV32A, quá trình kiểm thử không chỉ đánh giá kết quả tính toán của các lệnh nguyên tử mà còn kiểm tra khả năng đảm bảo tính nguyên tử khi truy cập bộ nhớ trong môi trường đa lõi.

Bên cạnh kiểm thử chức năng tập lệnh, hệ thống còn được kiểm tra khả năng xử lý các tình huống xung đột trong pipeline. Các trường hợp data hazard, load-use hazard và control hazard được xây dựng nhằm đánh giá hoạt động của cơ chế forwarding, stall và flush. Đây là các thành phần quan trọng giúp bộ xử lý duy trì kết quả thực thi chính xác khi các lệnh phụ thuộc dữ liệu hoặc thay đổi luồng điều khiển xuất hiện trong chương trình.

Ngoài ra, do hệ thống được mở rộng theo hướng đa lõi, quá trình kiểm thử còn hướng đến việc đánh giá khả năng hoạt động đồng thời của hai lõi RV32IMFA khi cùng truy cập bộ nhớ dùng chung. Các kịch bản kiểm thử đa lõi được xây dựng để kiểm tra cơ chế phân xử truy cập bus, khả năng cấp quyền truy cập cho từng lõi, cũng

như phản ứng của hệ thống khi hai lỗi phát sinh yêu cầu truy cập bộ nhớ tại cùng một thời điểm.

Cuối cùng, việc kiểm thử còn nhằm xác nhận khả năng triển khai thiết kế lên kit FPGA. Thông qua quá trình tổng hợp, nạp bitstream, nạp chương trình kiểm thử và quan sát kết quả thực thi, hệ thống được đánh giá ở mức thực nghiệm thay vì chỉ dừng lại ở mô phỏng. Kết quả kiểm thử là cơ sở để nhận xét mức độ đúng đắn, ổn định và khả năng mở rộng của bộ xử lý RV32IMFA trong hệ thống đa lỗi.

4.2. Môi trường và công cụ kiểm thử

Quá trình kiểm thử bộ xử lý RV32IMFA được thực hiện trên nhiều môi trường khác nhau nhằm đánh giá thiết kế ở cả mức mô phỏng và mức triển khai thực nghiệm. Việc kết hợp nhiều công cụ kiểm thử giúp đối chiếu kết quả thực thi của bộ xử lý với kết quả tham chiếu, từ đó xác định tính đúng đắn của từng nhóm lệnh cũng như toàn bộ hệ thống.

Trước hết, công cụ Spike được sử dụng làm mô hình tham chiếu cho kiến trúc RISC-V. Các chương trình kiểm thử được biên dịch và chạy trên Spike để thu được kết quả đúng mong đợi của từng lệnh hoặc từng nhóm lệnh. Kết quả này được dùng để so sánh với kết quả thực thi của bộ xử lý RV32IMFA trong quá trình mô phỏng và chạy thực nghiệm. Đối với các nhóm lệnh như RV32I, RV32M, RV32F và RV32A, Spike đóng vai trò quan trọng trong việc xác nhận giá trị thanh ghi, giá trị bộ nhớ và luồng thực thi chương trình.

Bên cạnh Spike, môi trường mô phỏng Verilog/SystemVerilog được sử dụng để kiểm tra hoạt động của từng khối chức năng trong thiết kế. Các testbench được xây dựng nhằm cung cấp xung nhịp, tín hiệu reset, chương trình kiểm thử và các điều kiện đầu vào cần thiết cho bộ xử lý. Thông qua mô phỏng, nhóm thực hiện có thể quan sát các tín hiệu bên trong hệ thống như giá trị PC, lệnh đang thực thi, dữ liệu thanh ghi, tín hiệu điều khiển pipeline, tín hiệu stall, flush và forwarding. Điều này giúp phát hiện lỗi thiết kế trước khi triển khai lên phần cứng.

Các chương trình kiểm thử được lưu dưới dạng file HEX để nạp vào bộ nhớ lệnh hoặc mô hình bộ nhớ trong testbench. File HEX chứa mã máy của các chương

trình kiểm thử đã được biên dịch từ mã assembly RISC-V. Khi mô phỏng, bộ nhớ lệnh đọc nội dung từ file HEX và cung cấp lệnh cho bộ xử lý thực thi. Cách làm này giúp quá trình kiểm thử linh hoạt hơn, vì có thể thay đổi chương trình kiểm thử mà không cần chỉnh sửa trực tiếp mã nguồn phần cứng.

Đối với kiểm thử phần cứng, công cụ Vivado được sử dụng để tổng hợp, ánh xạ, đặt tuyến và tạo bitstream cho thiết kế. Sau khi thiết kế phần cứng được tổng hợp thành công, bitstream được nạp lên kit FPGA để kiểm tra khả năng hoạt động thực tế của hệ thống. Trong quá trình này, các lỗi liên quan đến tài nguyên phần cứng, tín hiệu I/O, timing hoặc kết nối hệ thống có thể được phát hiện và điều chỉnh.

Ngoài Vivado, công cụ Vitis được sử dụng để tạo chương trình kiểm thử ở mức phần mềm và hỗ trợ quá trình nạp chương trình lên kit. Thông qua giao diện Vitis, chương trình có thể được biên dịch, chạy trên nền tảng phần cứng đã export từ Vivado và quan sát kết quả thông qua cửa sổ console. Các thông tin in ra console, trạng thái thực thi chương trình và giá trị kiểm tra được sử dụng làm bằng chứng cho quá trình kiểm thử thực nghiệm.

Như vậy, môi trường kiểm thử của đề tài bao gồm cả mô phỏng bằng testbench, kiểm chứng kết quả bằng Spike và triển khai thực tế trên kit FPGA thông qua Vivado/Vitis. Sự kết hợp này giúp quá trình đánh giá bộ xử lý RV32IMFA có độ tin cậy cao hơn, đồng thời cho phép kiểm tra thiết kế từ mức chức năng từng lệnh đến mức hệ thống hoàn chỉnh.

4.3. Kiểm thử chức năng các nhóm lệnh RV32IMFA

Sau khi xác định mục tiêu, môi trường và công cụ kiểm thử, nhóm thực hiện tiến hành xây dựng các chương trình kiểm thử cho từng nhóm lệnh thuộc tập lệnh RV32IMFA. Việc kiểm thử theo từng nhóm lệnh giúp đánh giá riêng biệt hoạt động của các khối chức năng trong bộ xử lý, đồng thời thuận tiện cho quá trình phát hiện và khoanh vùng lỗi khi kết quả thực thi không đúng như mong đợi.

Đối với mỗi nhóm lệnh, chương trình kiểm thử được viết dưới dạng mã assembly RISC-V, sau đó biên dịch thành mã máy và lưu dưới dạng file HEX để nạp vào bộ nhớ lệnh của hệ thống. Kết quả thực thi của bộ xử lý được so sánh với kết quả

tham chiếu từ Spike nhằm xác nhận tính đúng đắn của thiết kế. Các giá trị cần quan sát bao gồm giá trị thanh ghi, dữ liệu bộ nhớ, kết quả phép toán, trạng thái nhảy/rẽ nhánh và phản ứng của hệ thống trong các trường hợp đặc biệt.

4.3.1. Kiểm thử nhóm lệnh RV32I

Nhóm lệnh RV32I là tập lệnh số nguyên cơ bản của kiến trúc RISC-V, bao gồm các lệnh số học, logic, dịch bit, truy cập bộ nhớ, rẽ nhánh và nhảy. Đây là nhóm lệnh nền tảng trong quá trình kiểm thử vì hầu hết các thành phần cơ bản của bộ xử lý đều được sử dụng khi thực thi nhóm lệnh này, bao gồm ALU, tập thanh ghi số nguyên, bộ tạo giá trị tức thời, khối truy cập bộ nhớ và mạch cập nhật giá trị PC.

Trong quá trình kiểm thử, các lệnh số học và logic như ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT và SLTU được sử dụng để đánh giá hoạt động của khối ALU. Các lệnh immediate như ADDI, ANDI, ORI, XORI, SLLI, SRLI, SRAI, SLTI và SLTIU được dùng để kiểm tra khả năng xử lý toán hạng tức thời và quá trình mở rộng dấu. Bên cạnh đó, các lệnh load/store như LW, LH, LB, LHU, LBU, SW, SH và SB được đưa vào chương trình kiểm thử nhằm xác nhận khả năng đọc ghi dữ liệu giữa bộ xử lý và bộ nhớ. Các lệnh rẽ nhánh và nhảy như BEQ, BNE, BLT, BGE, BLTU, BGEU, JAL và JALR được sử dụng để kiểm tra khả năng thay đổi luồng thực thi chương trình.

Chương trình kiểm thử nhóm lệnh RV32I được mô phỏng trên Vivado để quan sát quá trình thực thi bên trong bộ xử lý. Trong quá trình mô phỏng, các tín hiệu như giá trị PC, mã lệnh đang thực thi, dữ liệu đọc từ tập thanh ghi, dữ liệu ghi về thanh ghi đích, tín hiệu truy cập bộ nhớ và kết quả xử lý của ALU được theo dõi nhằm đánh giá tính đúng đắn của thiết kế. Kết quả mô phỏng cho thấy bộ xử lý có thể thực hiện đúng trình tự các lệnh trong chương trình kiểm thử, đồng thời cập nhật dữ liệu thanh ghi và bộ nhớ phù hợp với từng loại lệnh.

Để tăng độ tin cậy của quá trình kiểm thử, cùng chương trình kiểm thử được chạy trên công cụ Spike nhằm tạo kết quả tham chiếu. Kết quả mô phỏng trên Vivado được đối chiếu với kết quả từ Spike thông qua giá trị thanh ghi và trạng thái bộ nhớ sau khi chương trình kết thúc. Việc kết quả giữa hai môi trường tương ứng nhau cho

thấy nhóm lệnh RV32I đã được thực thi đúng theo đặc tả, qua đó xác nhận hoạt động chính xác của các thành phần nền tảng trong bộ xử lý.

4.3.2. Kiểm thử nhóm lệnh RV32M

Nhóm lệnh RV32M mở rộng khả năng tính toán số nguyên của bộ xử lý thông qua các phép nhân, chia và lấy phần dư. Các lệnh được kiểm tra bao gồm MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM và REMU. So với các phép toán số học cơ bản trong nhóm RV32I, nhóm lệnh RV32M có độ phức tạp cao hơn do liên quan đến xử lý kết quả nhân có độ rộng lớn, phép chia có dấu, chia không dấu và các trường hợp đặc biệt trong quá trình tính toán.

Trong quá trình kiểm thử, các lệnh nhân được sử dụng để đánh giá khả năng xử lý phần thấp và phần cao của kết quả nhân. Lệnh MUL trả về 32 bit thấp của kết quả, trong khi các lệnh MULH, MULHSU và MULHU trả về 32 bit cao với các kiểu toán hạng khác nhau, bao gồm nhân có dấu, nhân không dấu và nhân giữa toán hạng có dấu với toán hạng không dấu. Việc kiểm tra các trường hợp này giúp đánh giá tính đúng đắn của khối MDU khi xử lý nhiều dạng dữ liệu số nguyên khác nhau.

Đối với các lệnh chia và lấy phần dư, chương trình kiểm thử sử dụng các lệnh DIV, DIVU, REM và REMU với cả toán hạng có dấu và không dấu. Ngoài các trường hợp tính toán thông thường, một số trường hợp biên như chia cho 0, chia số âm và phép chia với giá trị giới hạn của số nguyên cũng được đưa vào chương trình kiểm thử. Các trường hợp này nhằm đánh giá khả năng xử lý của bộ xử lý trong những tình huống đặc biệt, đồng thời kiểm tra việc ghi kết quả về thanh ghi đích sau khi khối MDU hoàn tất quá trình thực thi.

Chương trình kiểm thử nhóm lệnh RV32M được mô phỏng trên Vivado để theo dõi hoạt động của khối nhân chia trong bộ xử lý. Trong quá trình mô phỏng, các tín hiệu như toán hạng đầu vào, mã điều khiển phép toán, trạng thái xử lý của MDU, kết quả tính toán và dữ liệu ghi về thanh ghi đích được quan sát nhằm đánh giá tính đúng đắn của thiết kế. Kết quả mô phỏng cho thấy khối MDU có thể thực hiện các phép nhân, chia và lấy phần dư theo đúng trình tự, đồng thời trả kết quả phù hợp với từng loại lệnh.

Để kiểm chứng kết quả, cùng chương trình kiểm thử được chạy trên công cụ Spike nhằm tạo giá trị tham chiếu. Kết quả thu được từ mô phỏng Vivado được đối chiếu với kết quả từ Spike thông qua giá trị các thanh ghi sau khi chương trình kết thúc. Việc kết quả giữa hai môi trường tương ứng nhau cho thấy nhóm lệnh RV32M đã được thực thi đúng theo đặc tả, qua đó xác nhận khả năng hoạt động của khối nhân chia số nguyên trong bộ xử lý RV32IMFA.

4.3.3. Kiểm thử nhóm lệnh RV32F

Nhóm lệnh RV32F bổ sung khả năng xử lý số dấu chấm động độ chính xác đơn cho bộ xử lý. Việc kiểm thử nhóm lệnh này nhằm đánh giá hoạt động của khối FPU, tập thanh ghi dấu chấm động, các lệnh truy cập bộ nhớ dấu chấm động và các lệnh chuyển đổi dữ liệu giữa số nguyên và số thực. Đây là nhóm lệnh có độ phức tạp cao do liên quan đến biểu diễn số thực theo chuẩn IEEE 754, quá trình làm tròn, xử lý dấu và các trường hợp đặc biệt trong tính toán dấu chấm động.

Trong chương trình kiểm thử, các lệnh thuộc nhóm RV32F được sử dụng để đánh giá nhiều chức năng khác nhau của khối FPU. Các lệnh như FADD.S, FSUB.S, FMUL.S, FDIV.S và FSQRT.S được dùng để kiểm tra các phép toán số thực cơ bản. Các lệnh FMADD.S, FMSUB.S, FNMSUB.S và FNMADD.S được sử dụng để đánh giá nhóm lệnh fused multiply-add, trong đó phép nhân và phép cộng hoặc trừ được thực hiện trong cùng một thao tác. Bên cạnh đó, các lệnh so sánh như FEQ.S, FLT.S và FLE.S, các lệnh chọn giá trị như FMIN.S và FMAX.S, cũng như các lệnh gán dấu FSGNJ.S, FSGNJN.S và FSGNJX.S được đưa vào kiểm thử nhằm đánh giá khả năng xử lý dữ liệu dấu chấm động trong nhiều trường hợp khác nhau.

Ngoài các phép toán trực tiếp trên số thực, chương trình kiểm thử còn bao gồm các lệnh chuyển đổi và di chuyển dữ liệu như FCVT.W.S, FCVT.WU.S, FCVT.S.W, FCVT.S.WU, FMV.X.W, FMV.W.X và FCLASS.S. Các lệnh này dùng để kiểm tra khả năng trao đổi dữ liệu giữa tập thanh ghi số nguyên và tập thanh ghi dấu chấm động, đồng thời đánh giá việc phân loại các giá trị dấu chấm động đặc biệt. Các lệnh FLW và FSW cũng được sử dụng để kiểm tra quá trình đọc và ghi dữ liệu dấu chấm động giữa bộ nhớ và tập thanh ghi dấu chấm động.

Chương trình kiểm thử nhóm lệnh RV32F được mô phỏng trên Vivado để theo dõi hoạt động của khối FPU và các tín hiệu liên quan. Trong quá trình mô phỏng, các tín hiệu như dữ liệu đầu vào của FPU, mã điều khiển phép toán, trạng thái xử lý, kết quả đầu ra, tín hiệu ghi tập thanh ghi dấu chấm động và dữ liệu ghi về hệ thống được quan sát nhằm đánh giá tính đúng đắn của thiết kế. Kết quả mô phỏng cho thấy khối FPU có thể thực hiện các phép toán dấu chấm động, lệnh chuyển đổi và lệnh truy cập bộ nhớ dấu chấm động theo đúng trình tự.

Để kiểm chứng kết quả, cùng chương trình kiểm thử được chạy trên công cụ Spike nhằm tạo giá trị tham chiếu. Kết quả mô phỏng trên Vivado được đối chiếu với kết quả từ Spike thông qua giá trị thanh ghi số nguyên, thanh ghi dấu chấm động và trạng thái bộ nhớ sau khi chương trình kết thúc. Việc kết quả giữa hai môi trường tương ứng nhau cho thấy nhóm lệnh RV32F đã được thực thi đúng theo đặc tả, qua đó xác nhận khả năng hoạt động của khối FPU trong bộ xử lý RV32IMFA.

4.3.4. Kiểm thử nhóm lệnh RV32A

Nhóm lệnh RV32A hỗ trợ các thao tác nguyên tử trên bộ nhớ, bao gồm cặp lệnh LR/SC và các lệnh AMO. Đây là nhóm lệnh có vai trò quan trọng trong hệ thống đa lõi, vì nó giúp đảm bảo các thao tác đọc-sửa-ghi trên vùng nhớ dùng chung được thực hiện một cách nhất quán khi nhiều lõi cùng truy cập tài nguyên bộ nhớ. Việc kiểm thử nhóm lệnh này không chỉ đánh giá khả năng thực thi lệnh của từng lõi, mà còn liên quan trực tiếp đến khả năng đồng bộ dữ liệu trong môi trường hai lõi.

Các lệnh được kiểm tra bao gồm LR.W, SC.W, AMOSWAP.W, AMOADD.W, AMOAND.W, AMOOR.W, AMOXOR.W, AMOMAX.W, AMOMIN.W, AMOMAXU.W và AMOMINU.W. Đối với nhóm lệnh AMO, bộ xử lý cần đọc giá trị cũ từ bộ nhớ, thực hiện phép toán tương ứng với dữ liệu trong thanh ghi nguồn, sau đó ghi giá trị mới trở lại bộ nhớ. Đồng thời, giá trị cũ của bộ nhớ phải được ghi về thanh ghi đích theo đúng đặc tả của lệnh. Quá trình này giúp kiểm tra khả năng thực hiện thao tác đọc-sửa-ghi trong một chuỗi xử lý nhất quán.

Đối với cặp lệnh LR.W và SC.W, chương trình kiểm thử tập trung vào cơ chế đặt giữ chỗ địa chỉ và ghi có điều kiện. Lệnh LR.W được sử dụng để đọc dữ liệu từ

bộ nhớ và thiết lập trạng thái giữ chỗ đối với địa chỉ được truy cập. Sau đó, lệnh SC.W chỉ được ghi thành công nếu địa chỉ giữ chỗ vẫn còn hợp lệ và không bị thay đổi bởi thao tác ghi khác. Nếu điều kiện không thỏa mãn, lệnh SC.W phải trả về trạng thái thất bại. Đây là cơ chế quan trọng để hỗ trợ đồng bộ dữ liệu trong các hệ thống đa lõi.

Chương trình kiểm thử nhóm lệnh RV32A được mô phỏng trên Vivado để quan sát quá trình truy cập bộ nhớ, thực hiện thao tác nguyên tử và cập nhật dữ liệu sau khi lệnh hoàn tất. Trong quá trình mô phỏng, các tín hiệu như địa chỉ truy cập bộ nhớ, dữ liệu đọc từ bộ nhớ, dữ liệu ghi trở lại bộ nhớ, mã thao tác AMO, trạng thái reservation và dữ liệu ghi về thanh ghi đích được theo dõi nhằm đánh giá tính đúng đắn của thiết kế. Kết quả mô phỏng cho thấy bộ xử lý có thể thực hiện các thao tác nguyên tử theo đúng trình tự, đồng thời đảm bảo dữ liệu bộ nhớ được cập nhật phù hợp với từng loại lệnh.

Để kiểm chứng kết quả, cùng chương trình kiểm thử được chạy trên công cụ Spike nhằm tạo giá trị tham chiếu. Kết quả mô phỏng trên Vivado được đối chiếu với kết quả từ Spike thông qua giá trị thanh ghi và trạng thái bộ nhớ sau khi chương trình kết thúc. Việc kết quả giữa hai môi trường tương ứng nhau cho thấy nhóm lệnh RV32A đã được thực thi đúng theo đặc tả. Đặc biệt, kết quả kiểm thử nhóm lệnh này là cơ sở quan trọng để đánh giá khả năng hỗ trợ đồng bộ bộ nhớ và thao tác nguyên tử của hệ thống RV32IMFA trong môi trường đa lõi.

4.4. Kịch bản kiểm thử đa lõi

Bên cạnh việc kiểm thử từng nhóm lệnh độc lập, hệ thống còn được kiểm tra trong môi trường đa lõi nhằm đánh giá khả năng phối hợp giữa hai lõi RV32IMFA khi cùng truy cập tài nguyên bộ nhớ dùng chung. Trong thiết kế này, hai lõi xử lý có thể phát sinh yêu cầu đọc hoặc ghi bộ nhớ thông qua bus trung gian. Do đó, cần có cơ chế phân xử để quyết định lõi nào được cấp quyền truy cập tại một thời điểm, đồng thời đảm bảo dữ liệu trả về đúng lõi yêu cầu.

Mục tiêu của kiểm thử đa lõi là đánh giá hoạt động của khối phân xử bus, khả năng xử lý các yêu cầu truy cập đồng thời và tính đúng đắn của dữ liệu khi hai lõi

cùng thực thi chương trình. Các kịch bản kiểm thử được xây dựng từ đơn giản đến phức tạp, bao gồm truy cập tuần tự, truy cập đồng thời khác địa chỉ, truy cập đồng thời cùng địa chỉ và kiểm tra thao tác nguyên tử trong môi trường đa lõi.

4.4.1. Kiểm thử truy cập tuần tự

Ở kịch bản này, Core 0 và Core 1 lần lượt thực hiện truy cập bộ nhớ ở các thời điểm khác nhau. Mục đích của kiểm thử là xác nhận hoạt động của khối phân xử trong trường hợp chỉ có một lõi phát sinh yêu cầu truy cập tại một thời điểm. Đây là kịch bản cơ bản nhằm kiểm tra khả năng cấp quyền truy cập bộ nhớ của arbiter khi hệ thống không xảy ra tranh chấp tài nguyên.

Trong quá trình kiểm thử, Core 0 thực hiện thao tác đọc hoặc ghi bộ nhớ trước. Sau khi giao dịch của Core 0 hoàn tất, Core 1 mới phát sinh yêu cầu truy cập bộ nhớ. Khi chỉ có một yêu cầu hợp lệ xuất hiện trên bus, arbiter cần cấp quyền ngay cho lõi đang yêu cầu mà không tạo ra tín hiệu stall không cần thiết. Đồng thời, dữ liệu đọc hoặc ghi phải được truyền đúng giữa lõi xử lý và bộ nhớ dùng chung.

Kết quả kiểm thử cho thấy khi Core 0 và Core 1 truy cập bộ nhớ theo thứ tự tuần tự, arbiter có thể cấp quyền truy cập đúng cho từng lõi. Các giao dịch đọc và ghi được thực hiện đúng trình tự, tín hiệu phản hồi từ bộ nhớ được trả về đúng lõi yêu cầu và không xuất hiện xung đột trên bus dữ liệu. Điều này cho thấy cơ chế phân xử hoạt động ổn định trong trường hợp truy cập đơn giản, khi chỉ có một lõi sử dụng bộ nhớ tại một thời điểm.

4.4.2. Kiểm thử truy cập đồng thời khác địa chỉ

Ở kịch bản này, Core 0 và Core 1 cùng phát sinh yêu cầu truy cập bộ nhớ trong cùng một chu kỳ, nhưng địa chỉ truy cập của hai lõi là khác nhau. Đây là trường hợp dùng để kiểm tra khả năng xử lý của arbiter khi nhiều yêu cầu truy cập xuất hiện đồng thời trên bus. Mặc dù hai lõi truy cập đến hai địa chỉ khác nhau, hệ thống vẫn cần phân xử vì cả hai lõi cùng chia sẻ một đường truy cập bộ nhớ dùng chung.

Khi hai yêu cầu truy cập xuất hiện cùng lúc, arbiter sẽ lựa chọn một lõi được cấp quyền trước theo cơ chế phân xử đã thiết kế. Lõi còn lại sẽ tạm thời chờ cho đến khi bus sẵn sàng để thực hiện giao dịch tiếp theo. Trong thời gian chờ, hệ thống cần

đảm bảo lỗi chưa được cấp quyền không làm thay đổi dữ liệu trên bus và không gây ảnh hưởng đến giao dịch đang được phục vụ.

Kết quả kiểm thử cho thấy arbiter có khả năng lựa chọn đúng lỗi được phục vụ trước, đồng thời giữ lỗi còn lại ở trạng thái chờ thông qua tín hiệu điều khiển phù hợp. Sau khi giao dịch đầu tiên hoàn tất, arbiter tiếp tục cấp quyền cho lỗi còn lại để hoàn thành yêu cầu truy cập. Cả hai giao dịch đều được xử lý đúng, dữ liệu đọc hoặc ghi tương ứng với từng địa chỉ được đảm bảo chính xác. Điều này cho thấy hệ thống có khả năng xử lý các yêu cầu truy cập đồng thời khác địa chỉ mà không gây xung đột dữ liệu.

4.4.3. Kiểm thử truy cập đồng thời cùng địa chỉ

Kịch bản này được xây dựng nhằm đánh giá hoạt động của hệ thống trong trường hợp Core 0 và Core 1 cùng truy cập vào cùng một địa chỉ bộ nhớ. Đây là tình huống dễ phát sinh xung đột dữ liệu, đặc biệt khi có thao tác ghi hoặc thao tác đọc-sửa-ghi trên vùng nhớ dùng chung. Mục tiêu của kiểm thử là xác nhận rằng arbiter có thể điều phối thứ tự truy cập để tránh việc hai lõi cùng điều khiển bus tại cùng một thời điểm.

Trong quá trình kiểm thử, hai lõi cùng phát sinh yêu cầu truy cập đến cùng một địa chỉ bộ nhớ. Nếu cả hai lõi thực hiện thao tác đọc, hệ thống cần đảm bảo dữ liệu trả về cho từng lõi là chính xác. Nếu có thao tác ghi, arbiter phải đảm bảo tại một thời điểm chỉ có một lõi được quyền ghi dữ liệu vào bộ nhớ. Nhờ đó, giá trị cuối cùng trong bộ nhớ sẽ phản ánh đúng thứ tự các giao dịch đã được cấp quyền thực hiện.

Kết quả kiểm thử cho thấy các yêu cầu truy cập cùng địa chỉ được arbiter xử lý theo đúng thứ tự phân xử. Lõi được cấp quyền trước hoàn tất giao dịch trước, sau đó lõi còn lại mới tiếp tục thực hiện yêu cầu của mình. Không xuất hiện tình trạng hai lõi cùng ghi dữ liệu lên bus hoặc dữ liệu phản hồi bị trả về nhầm lõi. Kết quả này cho thấy hệ thống có khả năng duy trì tính đúng đắn của dữ liệu khi xảy ra tranh chấp truy cập bộ nhớ tại cùng một địa chỉ.

4.4.4. Kiểm thử thao tác nguyên tử trong môi trường đa lỗi

Đối với hệ thống có hỗ trợ mở rộng RV32A, việc kiểm thử thao tác nguyên tử trong môi trường đa lỗi là một nội dung quan trọng. Kịch bản này nhằm đánh giá khả năng thực hiện các lệnh như LR.W, SC.W, AMOADD.W, AMOSWAP.W, AMOXOR.W và các lệnh AMO khác khi hai lỗi cùng thao tác trên vùng nhớ dùng chung. Đây là nhóm kiểm thử có liên quan trực tiếp đến khả năng đồng bộ dữ liệu giữa các lỗi.

Trong quá trình kiểm thử, hai lỗi cùng thực hiện các thao tác đọc-sửa-ghi trên một địa chỉ bộ nhớ. Với các lệnh AMO, mỗi lỗi cần đọc giá trị cũ từ bộ nhớ, thực hiện phép toán tương ứng với dữ liệu trong thanh ghi nguồn, sau đó ghi giá trị mới trở lại bộ nhớ. Hệ thống phải đảm bảo toàn bộ chuỗi thao tác đọc-sửa-ghi của một lỗi được xử lý nhất quán, không bị xen ngang theo cách làm sai kết quả cuối cùng.

Đối với cặp lệnh LR.W và SC.W, một lỗi thực hiện LR.W để đặt trạng thái giữ chỗ trên một địa chỉ bộ nhớ. Sau đó, lệnh SC.W chỉ được ghi thành công nếu địa chỉ đang được giữ chỗ không bị lỗi khác ghi thay đổi trong khoảng thời gian giữa hai lệnh. Nếu có lỗi khác ghi vào cùng địa chỉ, lệnh SC.W phải trả về trạng thái thất bại. Cơ chế này giúp kiểm tra khả năng hỗ trợ đồng bộ bộ nhớ của hệ thống trong môi trường đa lỗi.

Kết quả kiểm thử cho thấy các thao tác nguyên tử được thực hiện đúng theo thứ tự cấp quyền của arbiter. Các lệnh AMO có thể đọc giá trị cũ, tính toán giá trị mới và cập nhật bộ nhớ theo đúng yêu cầu. Đối với cặp lệnh LR.W và SC.W, hệ thống có thể phân biệt trường hợp ghi thành công và thất bại dựa trên trạng thái giữ chỗ của địa chỉ bộ nhớ. Điều này cho thấy thiết kế có khả năng hỗ trợ các thao tác đồng bộ cơ bản trong môi trường hai lỗi.

Thông qua các kịch bản kiểm thử trên, có thể đánh giá khả năng hoạt động của hệ thống khi mở rộng từ một lỗi đơn sang mô hình hai lỗi. Kết quả kiểm thử cho thấy khối arbiter đóng vai trò quan trọng trong việc điều phối truy cập bộ nhớ, đảm bảo mỗi lỗi nhận đúng dữ liệu phản hồi và hạn chế xung đột khi nhiều lỗi cùng truy cập tài nguyên dùng chung. Đồng thời, việc kiểm thử các thao tác nguyên tử cho thấy hệ

thống có khả năng hỗ trợ đồng bộ dữ liệu trong các tình huống truy cập bộ nhớ phức tạp hơn.

4.5. Triển khai thực nghiệm bộ xử lý RV32IMFA trên kit KV260

Sau khi hoàn thành quá trình mô phỏng và kiểm thử chức năng trên Vivado, thiết kế bộ xử lý RV32IMFA được triển khai thực nghiệm trên kit KV260. Mục đích của bước này là kiểm tra khả năng tích hợp và hoạt động của hệ thống trên nền tảng phần cứng thực tế, đồng thời xác nhận chương trình kiểm thử có thể được nạp, thực thi và trả về kết quả thông qua môi trường Vitis.

Trong quá trình triển khai, thiết kế phần cứng sau khi được tổng hợp, ánh xạ, đặt tuyến và tạo bitstream trên Vivado được export sang Vitis dưới dạng hardware platform. Trên Vitis, chương trình kiểm thử được xây dựng nhằm giao tiếp với hệ thống RV32IMFA, nạp chương trình vào vùng bộ nhớ tương ứng và đọc lại kết quả sau khi thực thi. Việc triển khai này giúp đánh giá thiết kế ở mức thực nghiệm thay vì chỉ dừng lại ở mô phỏng chức năng.

Sau khi kết nối với kit KV260, chương trình kiểm thử được biên dịch và nạp xuống hệ thống thông qua Vitis. Quá trình nạp chương trình cho thấy nền tảng phần cứng được nhận diện đúng, thiết kế có thể giao tiếp với chương trình kiểm thử và quá trình thực thi được hoàn tất. Các thông tin trạng thái trong quá trình chạy chương trình được sử dụng để xác nhận rằng hệ thống đã được nạp thành công và có thể hoạt động trên phần cứng thực tế.

Sau khi chương trình được thực thi, các giá trị thanh ghi và dữ liệu kiểm thử được đọc lại nhằm phục vụ quá trình đánh giá. Các giá trị này được đối chiếu với kết quả mong đợi từ chương trình kiểm thử và kết quả tham chiếu từ mô phỏng. Việc kết quả thu được phù hợp với dữ liệu kiểm tra cho thấy chương trình đã tác động đúng đến vùng dữ liệu mong muốn, đồng thời bộ xử lý có thể thực thi chương trình trên nền tảng FPGA.

Kết quả thực nghiệm cho thấy thiết kế RV32IMFA có thể được triển khai trên kit KV260, chương trình kiểm thử được nạp thành công và hệ thống có khả năng trả về kết quả sau khi thực thi. Đây là cơ sở để chứng minh rằng bộ xử lý không chỉ hoạt

động trong môi trường mô phỏng mà còn có thể được tích hợp và kiểm tra trên nền tảng phần cứng FPGA thực tế.

4.6. Đánh giá hiệu năng hệ thống và kết quả kiểm thử chức năng

Sau khi hoàn thành các bước kiểm thử chức năng, kiểm thử đa lỗi, kiểm thử hazard control và triển khai thực nghiệm trên kit KV260, nhóm thực hiện tiến hành tổng hợp và đánh giá kết quả kiểm thử của hệ thống RV32IMFA. Mục tiêu của phần này là đưa ra nhận xét tổng quát về mức độ đáp ứng của thiết kế so với yêu cầu ban đầu, đồng thời đánh giá khả năng triển khai của bộ xử lý trên nền tảng FPGA thực tế.

Kết quả kiểm thử cho thấy bộ xử lý RV32IMFA có khả năng thực thi các nhóm lệnh chính thuộc kiến trúc RV32IMFA, bao gồm nhóm lệnh số nguyên cơ bản RV32I, nhóm lệnh nhân chia RV32M, nhóm lệnh dấu chấm động RV32F và nhóm lệnh nguyên tử RV32A. Các chương trình kiểm thử được chạy trên công cụ Spike để tạo kết quả tham chiếu, sau đó đối chiếu với kết quả mô phỏng trên Vivado và kết quả thực nghiệm trên kit KV260. Việc kết hợp nhiều môi trường kiểm thử giúp tăng độ tin cậy trong quá trình đánh giá thiết kế.

Đối với nhóm lệnh RV32I, các phép toán số học, logic, dịch bit, truy cập bộ nhớ, rẽ nhánh và nhảy đều cho kết quả đúng theo chương trình kiểm thử. Điều này cho thấy các khối cơ bản như ALU, bộ thanh ghi, bộ nhớ dữ liệu, khối tạo immediate và mạch cập nhật PC hoạt động đúng chức năng.

Đối với nhóm lệnh RV32M, các phép nhân, chia và lấy phần dư được kiểm tra với nhiều trường hợp khác nhau. Kết quả thu được cho thấy khối nhân chia có khả năng xử lý các phép toán số nguyên mở rộng và ghi kết quả đúng về thanh ghi đích. Các trường hợp biên như chia cho 0 hoặc phép toán với số âm cũng được đưa vào chương trình kiểm thử nhằm đánh giá tính ổn định của thiết kế.

Đối với nhóm lệnh RV32F, hệ thống kiểm tra hoạt động của khối FPU thông qua các phép toán dấu chấm động, lệnh chuyển đổi dữ liệu, lệnh so sánh và lệnh load/store dữ liệu dấu chấm động. Kết quả kiểm thử cho thấy bộ xử lý có khả năng thực hiện các thao tác dấu chấm động cơ bản, đồng thời hỗ trợ trao đổi dữ liệu giữa bộ nhớ, thanh ghi số nguyên và thanh ghi dấu chấm động.

Đối với nhóm lệnh RV32A, các lệnh nguyên tử như LR.W, SC.W và các lệnh AMO được kiểm tra nhằm đánh giá khả năng thao tác trên bộ nhớ dùng chung. Kết quả kiểm thử cho thấy hệ thống có khả năng thực hiện các thao tác đọc-sửa-ghi trên bộ nhớ và hỗ trợ cơ chế đồng bộ cần thiết trong môi trường đa lõi.

Bên cạnh kiểm thử từng nhóm lệnh, hệ thống còn được đánh giá thông qua các kịch bản kiểm thử đa lõi. Các trường hợp truy cập tuần tự, truy cập đồng thời khác địa chỉ, truy cập đồng thời cùng địa chỉ và thao tác nguyên tử trong môi trường hai lõi được sử dụng để kiểm tra khả năng phân xử truy cập bộ nhớ dùng chung. Kết quả kiểm thử cho thấy khối arbiter có thể điều phối yêu cầu từ hai lõi, đảm bảo mỗi lõi nhận đúng dữ liệu phản hồi và hạn chế xung đột khi cùng truy cập tài nguyên bộ nhớ. Kết quả tổng hợp của các nhóm kiểm thử chức năng và kiểm thử đa lõi được trình bày trong Bảng 4.1.

Bảng 4.1: Kết quả kiểm thử các nhóm chức năng của bộ xử lý RV32IMFA

Hạng mục kiểm thử	Nội dung kiểm thử	Kết quả đánh giá
RV32I	Số học, logic, load/store, branch, jump	Đạt
RV32M	Nhân, chia, lấy phần dư	Đạt
RV32F	Phép toán dấu chấm động, chuyển đổi, so sánh	Đạt
RV32A	LR/SC và các lệnh AMO	Đạt
Pipeline hazard	Forwarding, stall, flush	Đạt
Đa lõi	Phân xử truy cập bộ nhớ giữa hai lõi	Đạt
Triển khai phần cứng	Nạp và chạy chương trình trên kit KV260	Đạt

Bên cạnh kiểm thử chức năng, hệ thống cũng được đánh giá thông qua kết quả tổng hợp và triển khai trên Vivado. Các thông số như tài nguyên phần cứng sử dụng, tần số clock, timing và khả năng tạo bitstream được dùng để đánh giá mức độ phù hợp của thiết kế đối với nền tảng FPGA. Trong quá trình triển khai, hệ thống được cấu hình hoạt động với tần số clock 90 MHz. Đây là mức tần số được sử dụng để đánh giá khả năng hoạt động thực nghiệm của thiết kế trên kit KV260.

Kết quả triển khai thực nghiệm cho thấy thiết kế RV32IMFA có thể được tích hợp vào hệ thống phần cứng, tạo bitstream và nạp xuống kit KV260. Chương trình kiểm thử sau khi nạp có thể thực thi và trả về kết quả thông qua cửa sổ Terminal của Vitis. Đây là minh chứng cho thấy bộ xử lý không chỉ hoạt động ở mức mô phỏng mà còn có thể triển khai trên nền tảng FPGA thực tế.

Tuy nhiên, do hệ thống hỗ trợ nhiều thành phần phức tạp như FPU, khối nhân chia, nhóm lệnh nguyên tử và cấu trúc đa lỗi, thiết kế vẫn còn khả năng tiếp tục tối ưu trong các phiên bản sau. Một số hướng tối ưu có thể thực hiện bao gồm giảm độ trễ của các khối tính toán phức tạp, tối ưu tài nguyên phần cứng, cải thiện đường dữ liệu trong FPU và mở rộng thêm các chương trình kiểm thử tự động.

Nhìn chung, kết quả kiểm thử và triển khai cho thấy bộ xử lý RV32IMFA đã đáp ứng được các mục tiêu chính của đề tài. Thiết kế có khả năng thực thi các nhóm lệnh RV32IMFA, xử lý các tình huống hazard trong pipeline, hỗ trợ kiểm thử đa lỗi và có thể triển khai thực nghiệm trên kit KV260. Đây là cơ sở để khẳng định tính đúng đắn của thiết kế và khả năng phát triển tiếp trong các hệ thống xử lý RISC-V mở rộng.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Trong đề tài này, nhóm thực hiện đã nghiên cứu, thiết kế và kiểm thử bộ xử lý RISC-V có khả năng hỗ trợ tập lệnh RV32IMFA. Thiết kế được xây dựng theo kiến trúc pipeline, kết hợp các khối chức năng chính như khối nạp lệnh, giải mã lệnh, thực thi, truy cập bộ nhớ, ghi kết quả, bộ thanh ghi số nguyên, bộ thanh ghi dấu chấm động, ALU, MDU, FPU và các khối điều khiển hazard. Bên cạnh đó, hệ thống còn được mở rộng theo hướng đa lõi, cho phép hai lõi xử lý cùng truy cập tài nguyên bộ nhớ dùng chung thông qua cơ chế phân xử.

Về mặt chức năng, bộ xử lý đã được kiểm thử với các nhóm lệnh chính thuộc tập lệnh RV32IMFA. Nhóm lệnh RV32I được sử dụng để kiểm tra các phép toán số nguyên cơ bản, load/store, rẽ nhánh và nhảy. Nhóm lệnh RV32M được dùng để đánh giá khả năng thực hiện các phép nhân, chia và lấy phần dư. Nhóm lệnh RV32F được kiểm thử nhằm đánh giá hoạt động của khối xử lý dấu chấm động, các phép toán số thực, lệnh chuyển đổi và lệnh truy cập bộ nhớ dấu chấm động. Nhóm lệnh RV32A được kiểm tra thông qua các thao tác nguyên tử như LR/SC và các lệnh AMO, phục vụ cho môi trường đa lõi.

Ngoài kiểm thử chức năng từng nhóm lệnh, đề tài cũng đã xây dựng các kịch bản kiểm thử pipeline hazard nhằm đánh giá hoạt động của cơ chế forwarding, stall và flush. Các trường hợp data hazard, load-use hazard và control hazard được sử dụng để xác nhận khả năng xử lý xung đột trong quá trình thực thi pipeline. Kết quả kiểm thử cho thấy hệ thống có khả năng duy trì tính đúng đắn của chương trình khi các lệnh phụ thuộc dữ liệu hoặc thay đổi luồng điều khiển xuất hiện.

Đối với phần mở rộng đa lõi, nhóm thực hiện đã xây dựng các kịch bản kiểm thử truy cập bộ nhớ giữa hai lõi, bao gồm truy cập tuần tự, truy cập đồng thời khác địa chỉ, truy cập đồng thời cùng địa chỉ và thao tác nguyên tử trong môi trường đa lõi. Các kịch bản này giúp đánh giá khả năng phân xử truy cập bus, đảm bảo dữ liệu được phản hồi đúng lõi yêu cầu và hạn chế xung đột khi nhiều lõi cùng truy cập tài nguyên dùng chung.

Bên cạnh mô phỏng, thiết kế cũng được triển khai thực nghiệm trên kit KV260. Quá trình tổng hợp, tạo bitstream, export phần cứng sang Vitis và nạp chương trình kiểm thử lên kit đã được thực hiện thành công. Kết quả chạy chương trình được quan sát thông qua cửa sổ Terminal, cho thấy hệ thống có thể được nạp chương trình, thực thi và trả về kết quả kiểm thử trên phần cứng thực tế. Thiết kế được cấu hình hoạt động với tần số clock 90 MHz trong quá trình đánh giá thực nghiệm.

Nhìn chung, đề tài đã đạt được các mục tiêu chính đặt ra ban đầu. Bộ xử lý RV32IMFA đã được thiết kế, mô phỏng, kiểm thử theo từng nhóm chức năng, mở rộng theo hướng đa lõi và triển khai trên nền tảng FPGA thực tế. Kết quả đạt được là cơ sở để đánh giá tính đúng đắn của thiết kế, đồng thời tạo tiền đề cho các hướng cải tiến và phát triển tiếp theo.

5.2. Hạn chế của đề tài

Mặc dù thiết kế đã thực hiện được các chức năng chính, đề tài vẫn còn một số hạn chế nhất định. Thứ nhất, các khối xử lý phức tạp như FPU, đặc biệt là các phép chia và căn bậc hai dấu chấm động, có độ phức tạp phần cứng lớn và ảnh hưởng đáng kể đến timing của hệ thống. Do đó, các khối này vẫn cần được tối ưu thêm để nâng cao tần số hoạt động và giảm độ trễ đường dữ liệu.

Thứ hai, quá trình kiểm thử hiện tại chủ yếu tập trung vào các chương trình kiểm thử được xây dựng thủ công cho từng nhóm lệnh và từng kịch bản cụ thể. Mặc dù các test case này có thể đánh giá được chức năng chính của hệ thống, nhưng vẫn chưa bao phủ toàn bộ mọi trường hợp đặc biệt có thể xảy ra trong thực tế. Các trường hợp kiểm thử ngẫu nhiên, kiểm thử tự động hoặc kiểm thử với chương trình lớn hơn vẫn cần được mở rộng trong tương lai.

Thứ ba, phần đa lõi mới tập trung vào kiểm thử khả năng phân xử truy cập bộ nhớ và xử lý các thao tác nguyên tử cơ bản. Các vấn đề nâng cao hơn như cache coherence, tối ưu băng thông bộ nhớ, quản lý tranh chấp tài nguyên và đồng bộ giữa nhiều lõi vẫn chưa được triển khai đầy đủ.

Cuối cùng, việc triển khai trên kit KV260 đã chứng minh khả năng hoạt động thực nghiệm của thiết kế, tuy nhiên hệ thống vẫn cần thêm các bước tối ưu về tài

nguyên phần cứng, timing và hiệu năng nếu muốn phát triển thành một bộ xử lý có khả năng chạy các chương trình phức tạp hơn.

5.3. Hướng phát triển

Trong tương lai, đề tài có thể được tiếp tục phát triển theo hướng ưu tiên bổ sung hệ thống bộ nhớ đệm cache nhằm cải thiện hiệu năng truy cập bộ nhớ của bộ xử lý hai lõi. Mỗi lõi có thể được tích hợp cache lệnh và cache dữ liệu riêng để giảm số lần truy cập trực tiếp đến bộ nhớ dùng chung. Các tham số như dung lượng cache, kích thước dòng cache, số đường liên kết và chính sách thay thế cần được khảo sát để lựa chọn cấu hình phù hợp với tài nguyên FPGA và yêu cầu hiệu năng của hệ thống.

Khi hai lõi sử dụng các cache dữ liệu riêng, vấn đề nhất quán dữ liệu giữa các lõi cần được xem xét. Hệ thống có thể được bổ sung cơ chế theo dõi các giao dịch ghi, truyền tín hiệu snoop và vô hiệu hóa các dòng cache không còn hợp lệ. Ở giai đoạn tiếp theo, có thể nghiên cứu áp dụng các giao thức nhất quán cache như MSI hoặc MESI để bảo đảm hai lõi quan sát được dữ liệu đồng nhất khi cùng truy cập một vùng nhớ dùng chung. Cơ chế này cũng cần được phối hợp với các lệnh nguyên tử LR/SC và AMO nhằm duy trì tính đúng đắn trong các thao tác đồng bộ dữ liệu.

Khối Arbiter và giao tiếp AXI4 có thể được cải tiến để hỗ trợ hiệu quả hơn các yêu cầu phát sinh từ cache, bao gồm nạp dòng cache, ghi trả dữ liệu và xử lý nhiều giao dịch đang chờ. Hệ thống kiểm thử cũng cần được mở rộng với các kịch bản cache hit, cache miss, thay thế dòng, ghi trả dữ liệu và truy cập đồng thời từ hai lõi. Kết quả thực thi có thể được đối chiếu với mô hình tham chiếu để đánh giá tính đúng đắn và mức cải thiện hiệu năng sau khi tích hợp cache.

Việc bổ sung cache và cơ chế nhất quán dữ liệu sẽ giúp giảm độ trễ truy cập bộ nhớ, tăng hiệu suất thực thi và tạo nền tảng để mở rộng hệ thống lên nhiều lõi hơn. Đây là hướng phát triển quan trọng để đưa thiết kế từ mô hình bộ xử lý hai lõi cơ bản tiến gần hơn đến một hệ thống RISC-V đa lõi hoàn chỉnh trên FPGA.

TÀI LIỆU THAM KHẢO

A. Tài liệu tham khảo tiếng Việt

- [1] C. T. Bình và Đ. P. Hùng, “Thiết kế và hiện thực lõi vi xử lý RISC-V RV32IF hỗ trợ 4-Way Set Associative Cache và bộ tạo số ngẫu nhiên thực trên FPGA,” 2023.
- [2] T. T. P. Nam và N. Đ. D. Khang, “Hiện thực một NoC sử dụng lõi vi xử lý RISC-V RV32IMF thông qua giao thức TileLink trên FPGA,” 2024.
- [3] N. P. H. Phú, “Research design of a multicore processor based on the RISC-V instruction set architecture,” 2021.
- [4] K. T. Tài và P. T. Đức, “Thiết kế bộ vi xử lý hai lõi dựa trên kiến trúc tập lệnh RISC-V RV32ICMFA,” 2025.
- [5] Đ. T. Thuận, “Hiện thực khối floating-point tích hợp vào bộ xử lý RISC-V 32-bit,” 2024.

B. Tài liệu tham khảo tiếng Anh

- [6] A. Waterman và K. Asanović, The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, RISC-V International.
- [7] A. Waterman et al., The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, RISC-V International.
- [8] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition. Cambridge, MA, USA: Morgan Kaufmann, 2017.
- [9] Domipheus Labs, “Designing a RISC-V CPU in VHDL, Part 20: Interrupts and Exceptions,” 2020.
- [10] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
- [11] R. Holler, D. Haselberger, D. Ballek, P. Rossler, M. Krapfenbauer, và M. Linauer, “Open-Source RISC-V Processor IP Cores for FPGAs – Overview and Evaluation,” 2019.

- [12] S. Pinto and C. Garlati, “User Mode Interrupts: A Must for Securing Embedded Systems,” in Embedded World Conference, Feb. 2019.
- [13] Y. Sun et al., “DuckCore: A Fault-Tolerant Processor Core Architecture Based on the RISC-V ISA,” Electronics, 2022.
- [14] Y. Sarkar and V. K. S. Patel, “32-Bit RISC-V Pipelined Dual Core Processor: Design and Optimization,” *International Journal of Creative Research Thoughts*, 2024.
- [15] Z. Zuckerman, P. Mantovani, D. Giri, and L. P. Carloni, “Enabling Heterogeneous, Multicore SoC Research with RISC-V and ESP,” *arXiv preprint*, 2022.

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

TP. Hồ Chí Minh, ngày 15 tháng 07 năm 2026

**BẢNG GIẢI TRÌNH CHỈNH SỬA
BÁO CÁO KHOA LUẬN TỐT NGHIỆP SAU BẢO VỆ**

Số quyết định Hội đồng: 710/QĐ-ĐHCNTT ngày 19 tháng 06 năm 2026

Ngày bảo vệ: 06/07/2026

Tên đề tài: THIẾT KẾ BỘ VI XỬ LÝ 2 LỖI RISC-V DỰA TRÊN KIẾN TRÚC TẬP LỆNH
RV32IMFA

Danh sách sinh viên

Sinh viên 1: Nguyễn Đăng Thanh Tuệ.

MSSV: 22251616

Sinh viên 2:

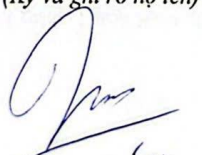
MSSV:

Giảng viên hướng dẫn: Th.S Thân Thế Tùng, K.S Trần Đại Dương

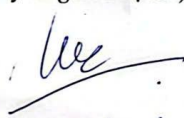
Căn cứ trên nội dung Biên bản Hội đồng khóa luận tốt nghiệp ngày 06/07/2026 và các góp ý của các thành viên Hội đồng, nhóm sinh viên thực hiện khóa luận xin phép được giải trình nội dung cập nhật báo cáo chi tiết như sau:

STT	Nội dung góp ý của Hội đồng	Nội dung giải trình cập nhật báo cáo	Trang
1	Giải thích chiến lược xử lý bus khi hai lỗi truy xuất cùng địa chỉ và truy xuất hai địa chỉ khác nhau.	Đã bổ sung nội dung mô tả cơ chế phân xử truy cập bộ nhớ dùng chung giữa hai lỗi. Báo cáo trình bày nguyên lý hoạt động của khối Arbiter trong hai trường hợp: hai lỗi đồng thời truy cập các địa chỉ khác nhau và hai lỗi đồng thời truy cập cùng một địa chỉ.	58-60
2	Nếu tiếp tục phát triển đề tài thì sinh viên muốn phát triển theo hướng nào.	Đã bổ sung mục Định hướng phát triển của đề tài. Nội dung tập trung vào việc tích hợp bộ nhớ đệm (cache) cho từng lõi xử lý, nghiên cứu cơ chế đảm bảo tính nhất quán cache (cache coherence) trong hệ thống đa lõi và mở rộng hệ thống kiểm thử nhằm đánh giá hiệu năng sau khi tích hợp cache.	67

Xác nhận của GVHD
(Ký và ghi rõ họ tên)


Thân Thế Tùng

Sinh viên thực hiện
(Ký và ghi rõ họ tên)


Nguyễn Đăng Thanh Tuệ

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

TP. Hồ Chí Minh, ngày 15 tháng 7 năm 2026

DANH MỤC KIỂM TRA BÁO CÁO KLTN SAU BẢO VỆ

STT	Nội dung kiểm tra	Tình trạng kiểm tra	
		Đã thực hiện	Chưa thực hiện
1	Bìa báo cáo theo mẫu, hướng dẫn	TRUE	
2	Kiểm tra mục lục, danh mục hình ảnh, danh mục bảng biểu, ký hiệu viết tắt	TRUE	
3	Có tóm tắt bằng tiếng Việt và tiếng Anh	TRUE	
4	Hình ảnh và bảng biểu có đầy đủ tên, đánh số thứ tự, phân tích đầy đủ	TRUE	
5	Báo cáo có đánh số trang theo quy định	TRUE	
6	Các chương được bắt đầu bằng một trang mới	TRUE	
7	Kiểm tra, chỉnh sửa lỗi chính tả, kiểu chữ và font chữ	TRUE	
8	Tài liệu tham khảo được trích dẫn đầy đủ theo quy định	TRUE	
9	Tài liệu tham khảo được trích dẫn đầy đủ theo quy định	TRUE	
10	Có giải trình chỉnh sửa theo góp ý của hội đồng	TRUE	

Xác nhận của GVHD
(Ký và ghi rõ họ tên)


Trần Thế Tung